Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- Recurrent Neural Networks
- RNNs as Language Models
- RNNs for other NLP tasks
- Stacked and Bidirectional RNN architectures
- The LSTM
- Summary: Common RNN NLP Architectures

Recurrent Neural Network (RNN)

- RNNs have a mechanism that deals directly with the sequential nature of language, allowing them to handle the temporal nature of language without the use of arbitrary fixed-sized windows. The recurrent network offers a new way to represent the prior context, in its recurrent connections, allowing the model's decision to depend on information from hundreds of words in the past

Recurrent neural network (RNN)

- A recurrent neural network (RNN) is any network that contains a cycle within its network connections, meaning that the value of some unit is directly, or indirectly, dependent on its own earlier outputs as an input.

- we consider a class of recurrent networks referred to as Elman Networks (Elman, 1990) or simple recurrent net. These networks are useful in their own right and serve as the basis for more complex approaches like the Long Short-Term Memory (LSTM) networks.

RNN

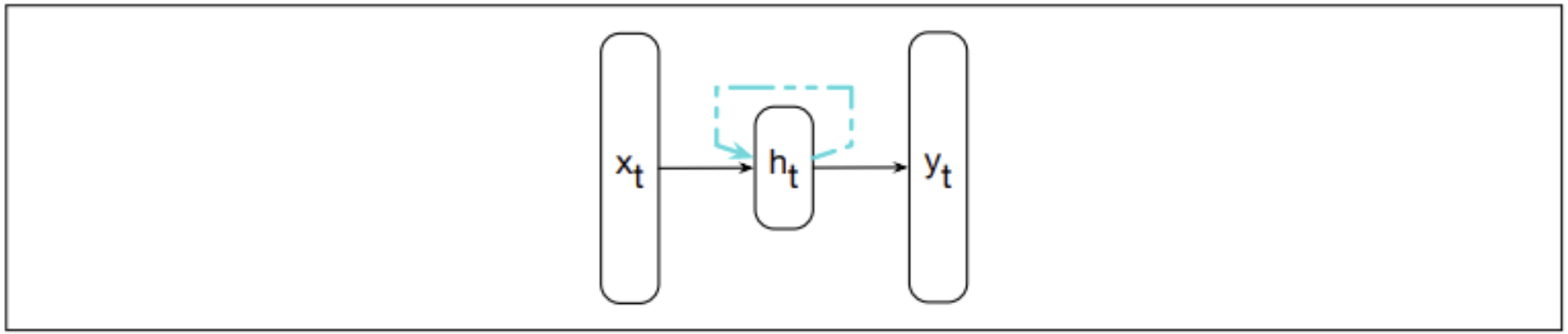
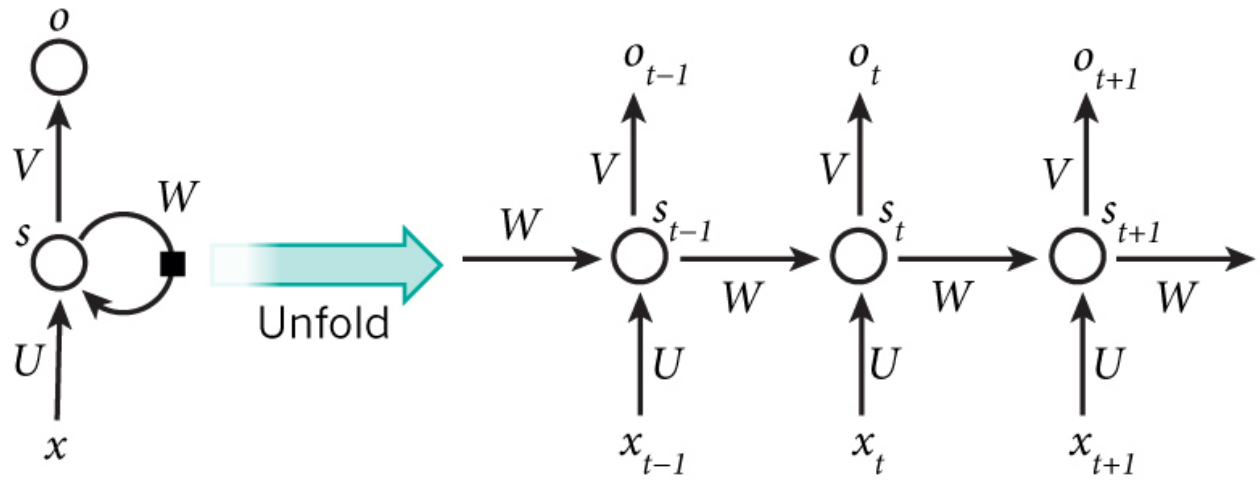
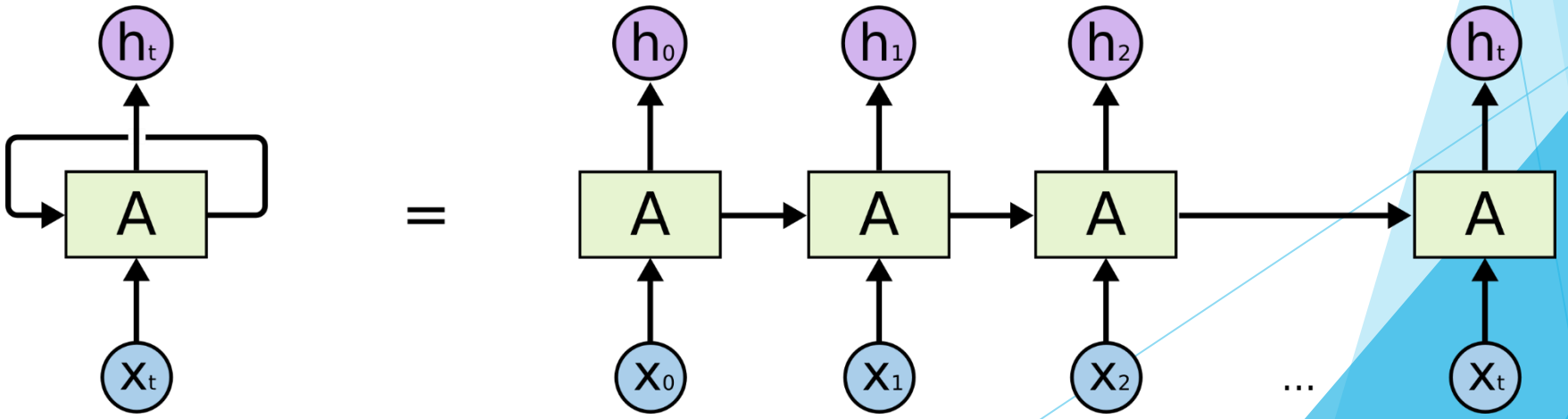


Figure 9.1 Simple recurrent neural network after Elman (1990). The hidden layer includes a recurrent connection as part of its input. That is, the activation value of the hidden layer depends on the current input as well as the activation value of the hidden layer from the previous time step.

Recurrent Neural Network





RNN

- As with ordinary feedforward networks, an input vector representing the current input, Xt , is multiplied by a weight matrix and then passed through a non-linear activation function to compute the values for a layer of hidden units. This hidden layer is then used to calculate a corresponding output, Yt. Here, sequences are processed by presenting one item at a time to the network. We'll use subscripts to represent time, thus Xt will mean the input vector X at time t. The key difference from a feedforward network lies in the recurrent link shown in the figure with the dashed line. This link augments the input to the computation at the hidden layer with the value of the hidden layer from the preceding point in time.

RNN

- The hidden layer from the previous time step provides a form of memory, or context, that encodes earlier processing and informs the decisions to be made at later points in time. Critically, this approach does not impose a fixed-length limit on this prior context; the context embodied in the previous hidden layer can include information extending back to the beginning of the sequence.
- Adding this temporal dimension makes RNNs appear to be more complex than non-recurrent architectures. But in reality, they're not all that different. Given an input vector and the values for the hidden layer from the previous time step, we're still performing the standard feedforward calculation.

RNN

- The most significant change lies in the new set of weights U, that connect the hidden layer from the previous time step to the current hidden layer. These weights determine how the network makes use of past context in calculating the output for the current input. As with the other weights in the network, these connections are trained via backpropagation.

Inference in RNNs

- Forward inference (mapping a sequence of inputs to a sequence of outputs) in an RNN is nearly identical to what we've already seen with feedforward networks. To compute an output Yt for an input Xt , we need the activation value for the hidden layer ht . To calculate this, we multiply the input Xt with the weight matrix W, and the hidden layer from the previous time step ht−1 with the weight matrix U. We add these values together and pass them through a suitable activation function, g, to arrive at the activation value for the current hidden layer, ht. Once we have the values for the hidden layer, we proceed with the usual computation to generate the output vector

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t) \quad (9.1)$$

$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t) \quad (9.2)$$

Inference in RNNs

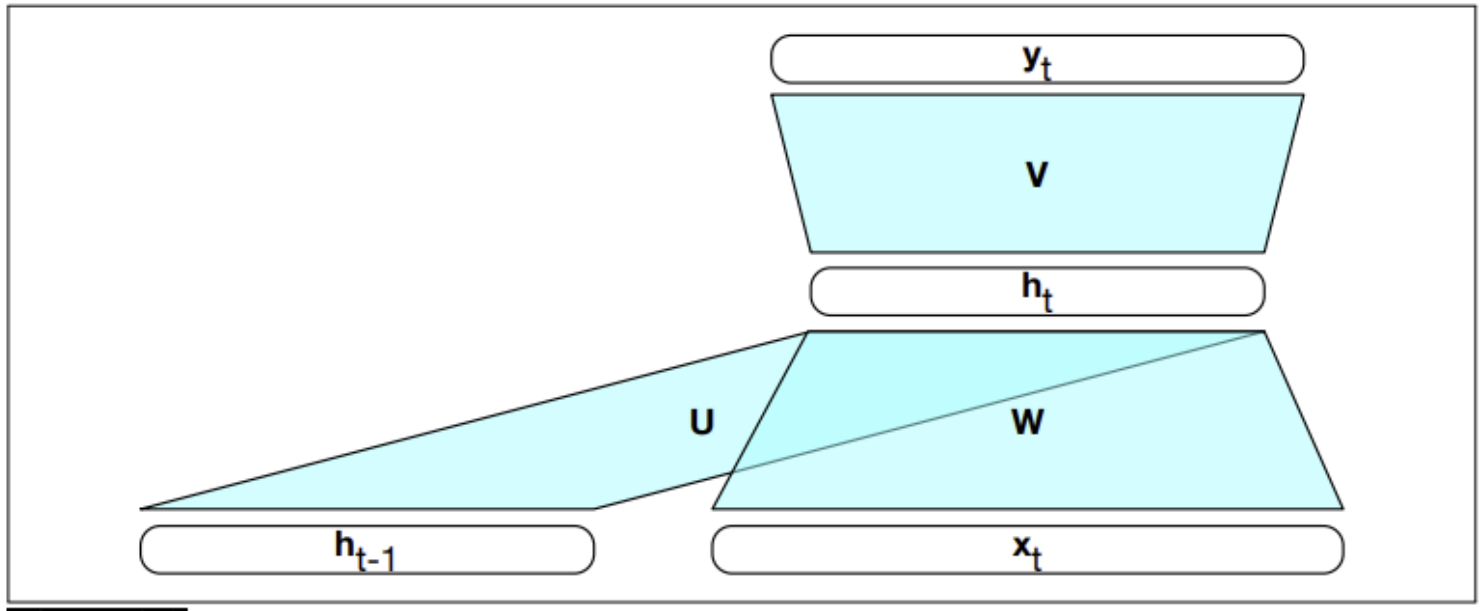


Figure 9.2 Simple recurrent neural network illustrated as a feedforward network.

Inference in RNNs

- be careful about specifying the dimensions of the input, hidden and output layers, as well as the weight matrices to make sure these calculations are correct. Let's refer to the input, hidden and output layer dimensions as din, dh, and dout respectively. Given this, our three parameter matrices are: $W \in \mathbb{R}^{d_h \times d_{in}}$, $\mathbf{U} \in \mathbb{R}^{d_h \times d_h}$, and $\mathbf{V} \in \mathbb{R}^{d_{out} \times d_h}$. . In the commonly encountered case of soft classification, computing Yt consists of a softmax computation that provides a probability distribution over the possible output classes.

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t) \qquad (9.3)$$

- The fact that the computation at time t requires the value of the hidden layer from time t −1 mandates an incremental inference algorithm that proceeds from the start of the sequence to the end as illustrated in Fig. 9.3

function FORWARDRNN(**x**, *network*) **returns** output sequence **y**

$\mathbf{h}_0 \leftarrow 0$
for $i \leftarrow 1$ **to** LENGTH(**x**) **do**
  $\mathbf{h}_i \leftarrow g(\mathbf{U}\mathbf{h}_{i-1} + \mathbf{W}\mathbf{x}_i)$
  $\mathbf{y}_i \leftarrow f(\mathbf{V}\mathbf{h}_i)$
return y

Figure 9.3 Forward inference in a simple recurrent network. The matrices **U**, **V** and **W** are shared across time, while new values for **h** and **y** are calculated with each time step.

- The sequential nature of simple recurrent networks can be seen by unrolling the network in time as is shown in Fig. 9.4. In this figure, the various layers of units are copied for each time step to illustrate that they will have differing values over time. However, the various weight matrices are shared across time.

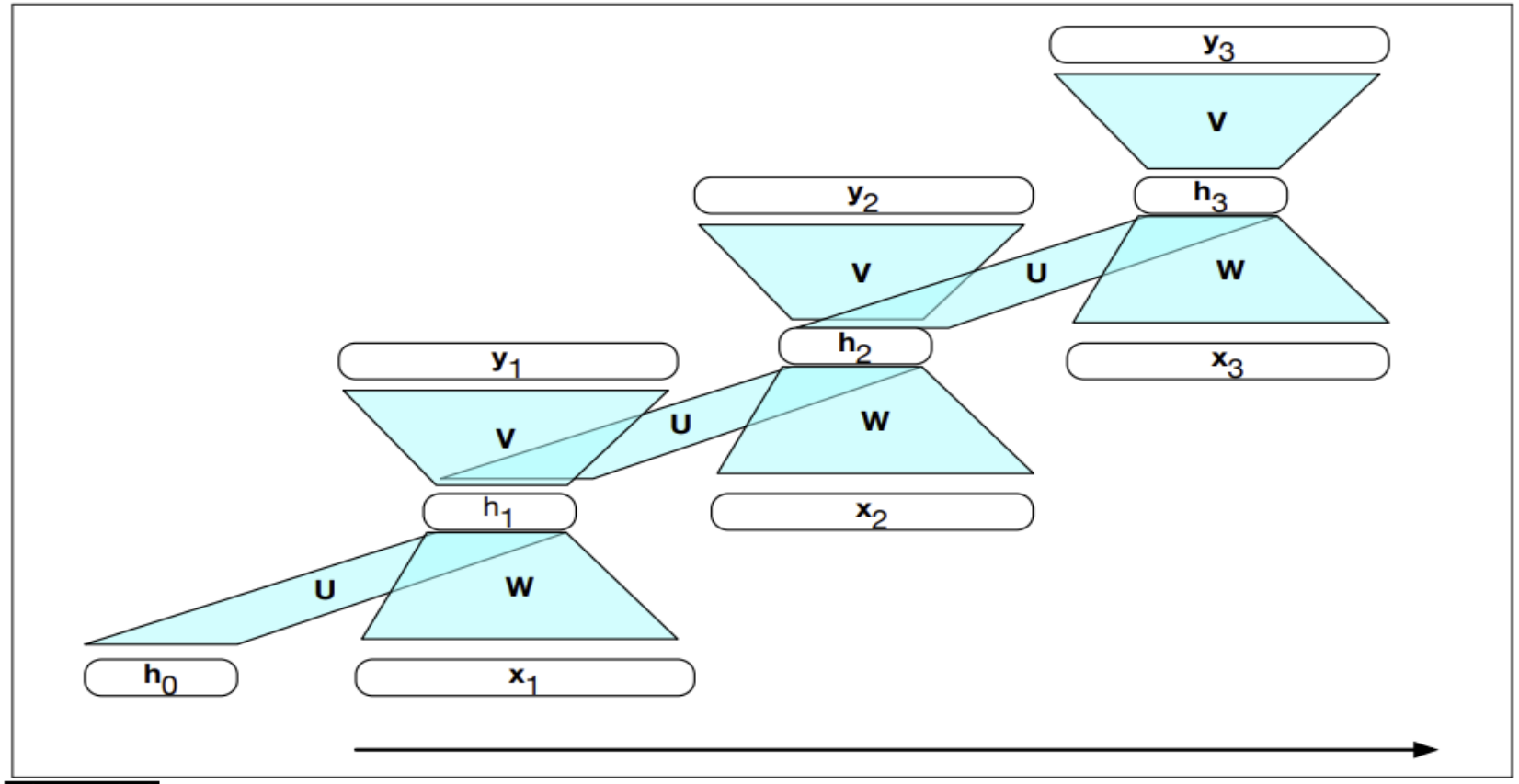


Figure 9.4 A simple recurrent neural network shown unrolled in time. Network layers are recalculated for each time step, while the weights **U**, **V** and **W** are shared across all time steps.

Training

- As with feedforward networks, we'll use a training set, a loss function, and backpropagation to obtain the gradients needed to adjust the weights in these recurrent networks.

- As shown in Fig. 9.2, we now have 3 sets of weights to update: W, the weights from the input layer to the hidden layer, U, the weights from the previous hidden layer to the current hidden layer, and finally V, the weights from the hidden layer to the output layer.

Training an RNN as language model

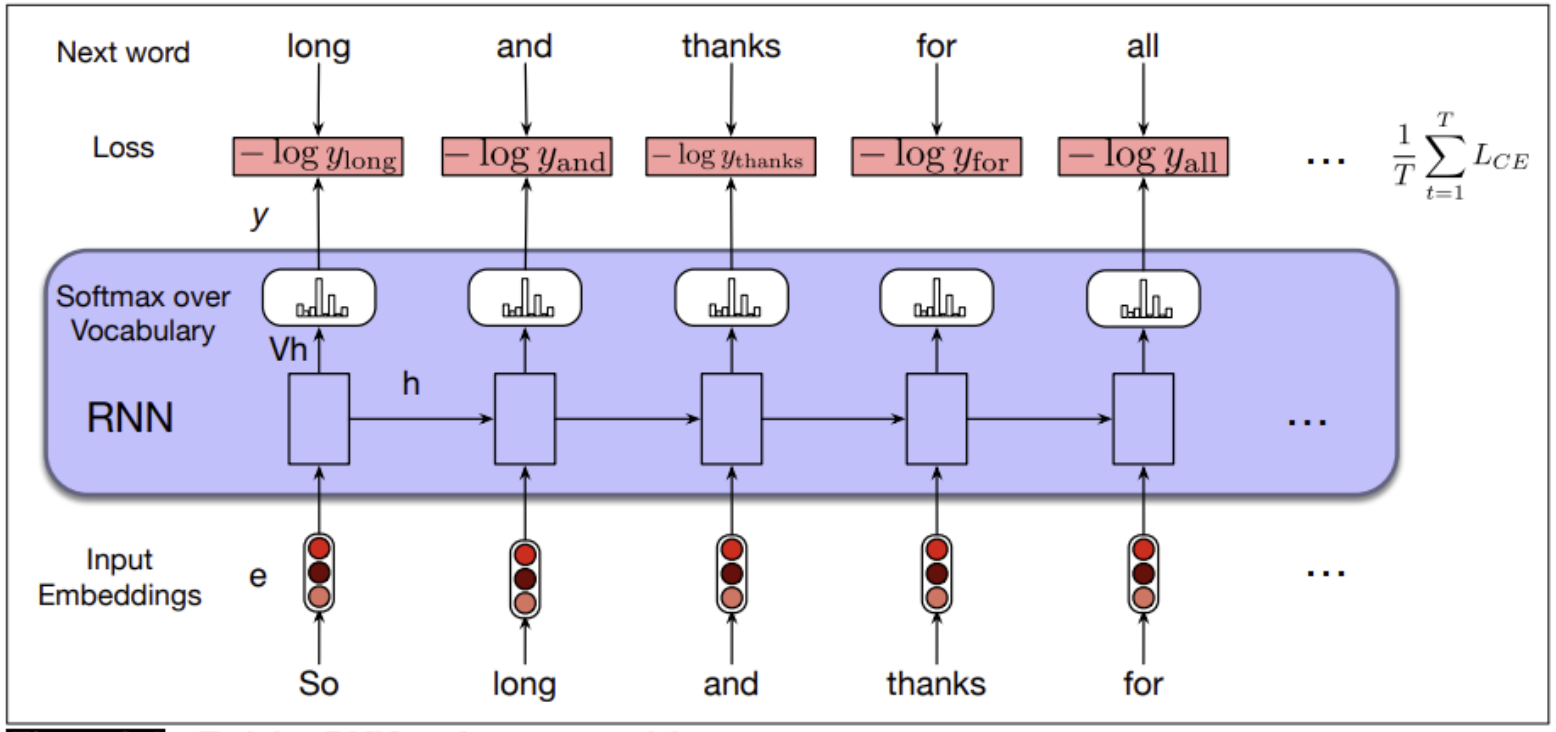


Figure 9.6 Training RNNs as language models.

In the case of language modeling, the correct distribution Yt comes from knowing the next word. This is represented as a one-hot vector corresponding to the vocabulary

Training an RNN as language model

- where the entry for the actual next word is 1, and all the other entries are 0. Thus, the cross-entropy loss for language modeling is determined by the probability the model assigns to the correct next word. So at time t the CE loss is the negative log probability the model assigns to the next word in the training sequence.

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}] \qquad (9.11)$$

Weight Tying

The input embedding matrix E and the final layer matrix V, which feeds the output softmax. The columns of E represent the word embeddings for each word in the vocabulary learned during the training process with the goal that words that have similar meaning and function will have similar embeddings. And, since the length of these embeddings corresponds to the size of the hidden layer dh, the shape of the embedding matrix E is dh ×|V|.

Weight Tying

- The final layer matrix V provides a way to score the likelihood of each word in the vocabulary given the evidence present in the final hidden layer of the network through the calculation of Vh. This results in dimensionality |V| ×dh. That is, the rows of V provide a second set of learned word embeddings that capture relevant aspects of word meaning. Weight tying is a method that dispenses with this redundancy and simply uses a single set of embeddings at the input and softmax layers. That is, we dispense with V and use E in both the start and end of the computation.

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t \tag{9.12}$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t) \tag{9.13}$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{E}^\intercal \mathbf{h}_t) \tag{9.14}$$

RNNs for other NLP tasks

- Apply RNN to three types of NLP tasks: sequence classification tasks like sentiment analysis and topic classification, sequence labeling tasks like part-of-speech tagging, and text generation tasks, including with a new architecture called the encoder-decoder.

- **Sequence Labeling** : the network's task is to assign a label chosen from a small fixed set of labels to each element of a sequence, like the part-of-speech tagging and named entity recognition tasks.

- In an RNN approach to sequence labeling, inputs are word embeddings and the outputs are tag probabilities generated by a softmax layer over the given tagset, as illustrated in Fig. 9.7.

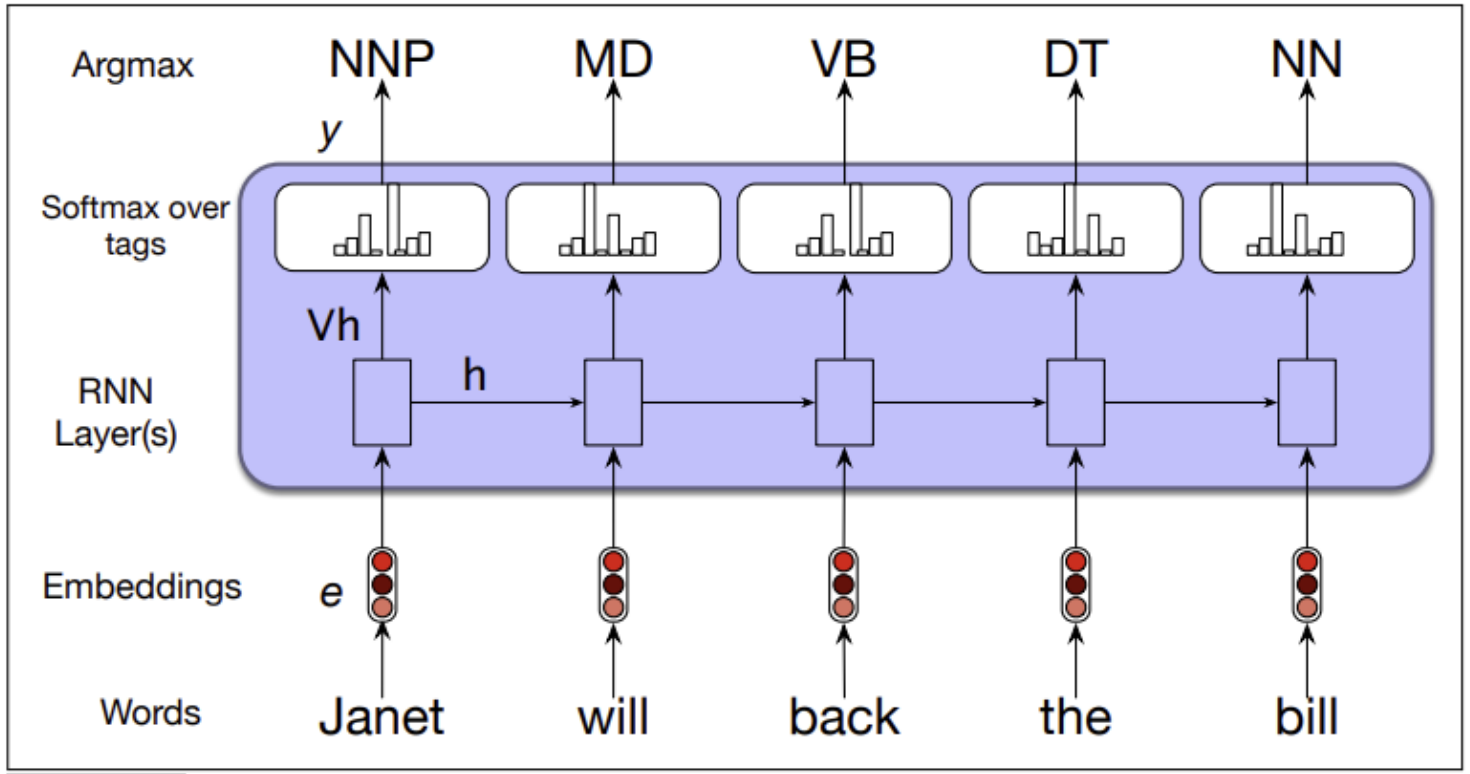


Figure 9.7 Part-of-speech tagging as sequence labeling with a simple RNN. Pre-trained word embeddings serve as inputs and a softmax layer provides a probability distribution over the part-of-speech tags as output at each time step.

Parts of speech tagging

- In this figure, the inputs at each time step are pretrained word embeddings corresponding to the input tokens. The RNN block is an abstraction that represents an unrolled simple recurrent network consisting of an input layer, hidden layer, and output layer at each time step, as well as the shared U, V and W weight matrices that comprise the network.

Parts of speech tagging

- The outputs of the network at each time step represent the distribution over the POS tagset generated by a softmax layer. To generate a sequence of tags for a given input, we run forward inference over the input sequence and select the most likely tag from the softmax at each step. Since we're using a softmax layer to generate the probability distribution over the output tagset at each time step.

RNNs for Sequence Classification

- This is the set of tasks commonly called text classification, like sentiment analysis or spam detection, in which we classify a text into two or three classes as well as classification tasks with a large number of categories, like document-level topic classification, or message routing for customer service applications.
- To apply RNNs in this setting, we pass the text to be classified through the RNN a word at a time generating a new hidden layer at each time step. We can then take the hidden layer for the last token of the text, hn, to constitute a compressed representation of the entire sequence. We can pass this representation hn to a feedforward network that chooses a class via a softmax over the possible classes

RNNs for Sequence Classification

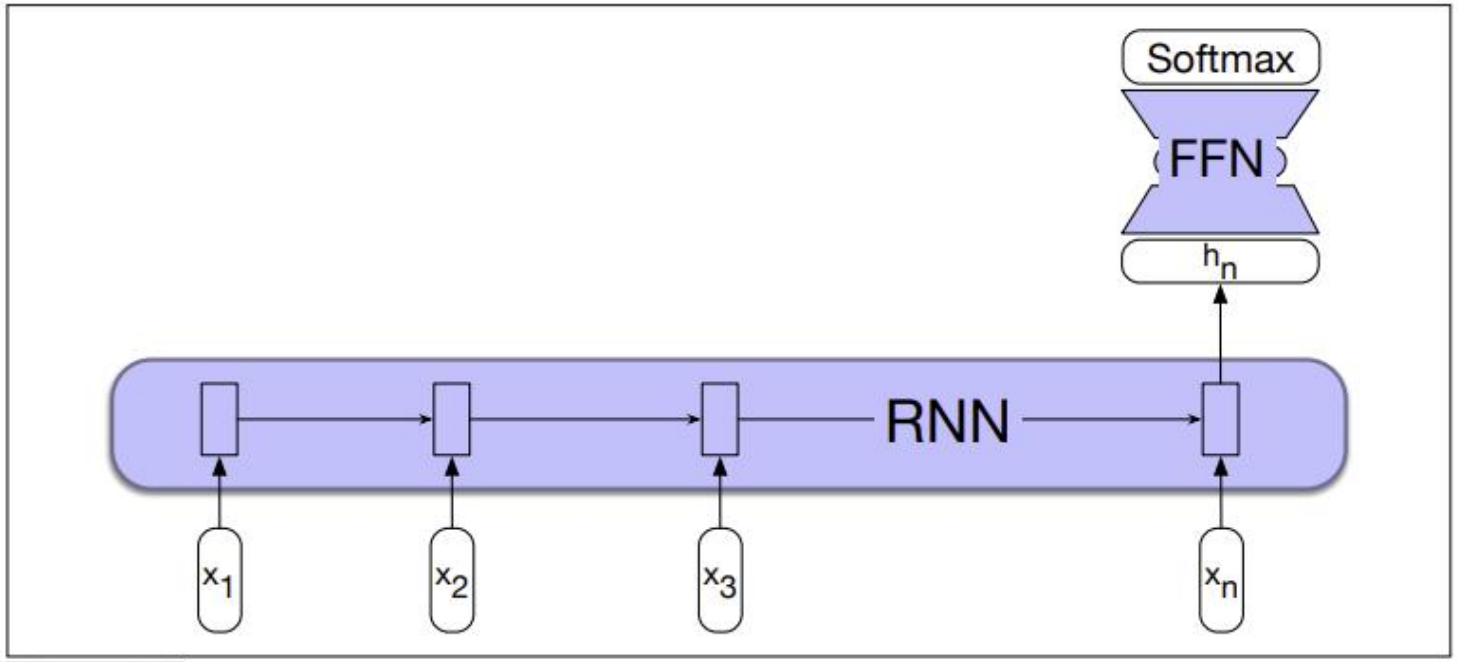


Figure 9.8 Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

RNNs for Sequence Classification

- Note that in this approach we don't need intermediate outputs for the words in the sequence preceding the last element. Therefore, there are no loss terms associated with those elements. Instead, the loss function used to train the weights in the network is based entirely on the final text classification task. The output from the softmax output from the feedforward classifier together with a cross-entropy loss drives the training.

Generation with RNN-Based Language Models

- Text generation is of enormous practical importance, part of tasks like question answering, machine translation, text summarization, grammar correction, story generation, and conversational dialogue; any task where a system needs to produce text, conditioned on some other text.

- Shannon (Shannon, 1951) and the psychologists George Miller and Jennifer Selfridge (Miller and Selfridge, 1950). We first randomly sample a word to begin a sequence based on its suitability as the start of a sequence. We then continue to sample words conditioned on our previous choices until we reach a pre-determined length, or an end of sequence token is generated.

Generation with RNN-Based Language Models

- Technically an autoregressive model is a model that predicts a value at time t based on a linear function of the previous values at times t −1, t −2, and so on. Although language models are not linear (since they have many layers of non-linearities), we loosely refer to this generation technique as autoregressive generation since the word generated at each time step is conditioned on the word selected by the network from the previous step. Fig. 9.9 illustrates this approach. In this figure, the details of the RNN's hidden layers and recurrent connections are hidden within the blue block

Generation with RNN-Based Language Models

- This simple architecture underlies state-of-the-art approaches to applications such as machine translation, summarization, and question answering.

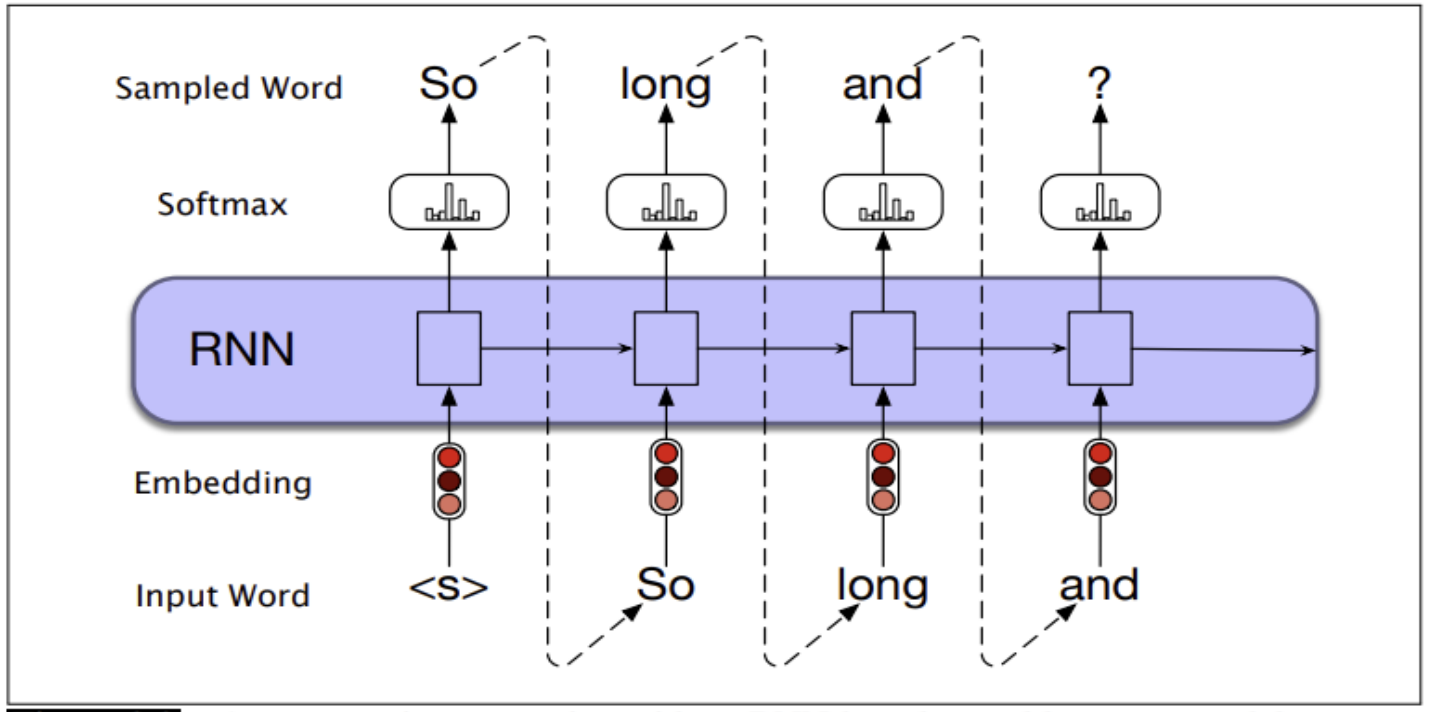


Figure 9.9 Autoregressive generation with an RNN-based neural language model.

RNN types

- Stacked RNN

- In our examples thus far, the inputs to our RNNs have consisted of sequences of word or character embeddings (vectors) and the outputs have been vectors useful for predicting words, tags or sequence labels. However, nothing prevents us from using the entire sequence of outputs from one RNN as an input sequence to another one Stacked RNNs consist of multiple networks where the output of one layer serves as the input to a subsequent layer, as shown in Fig. 9.10.

Stacked RNNs

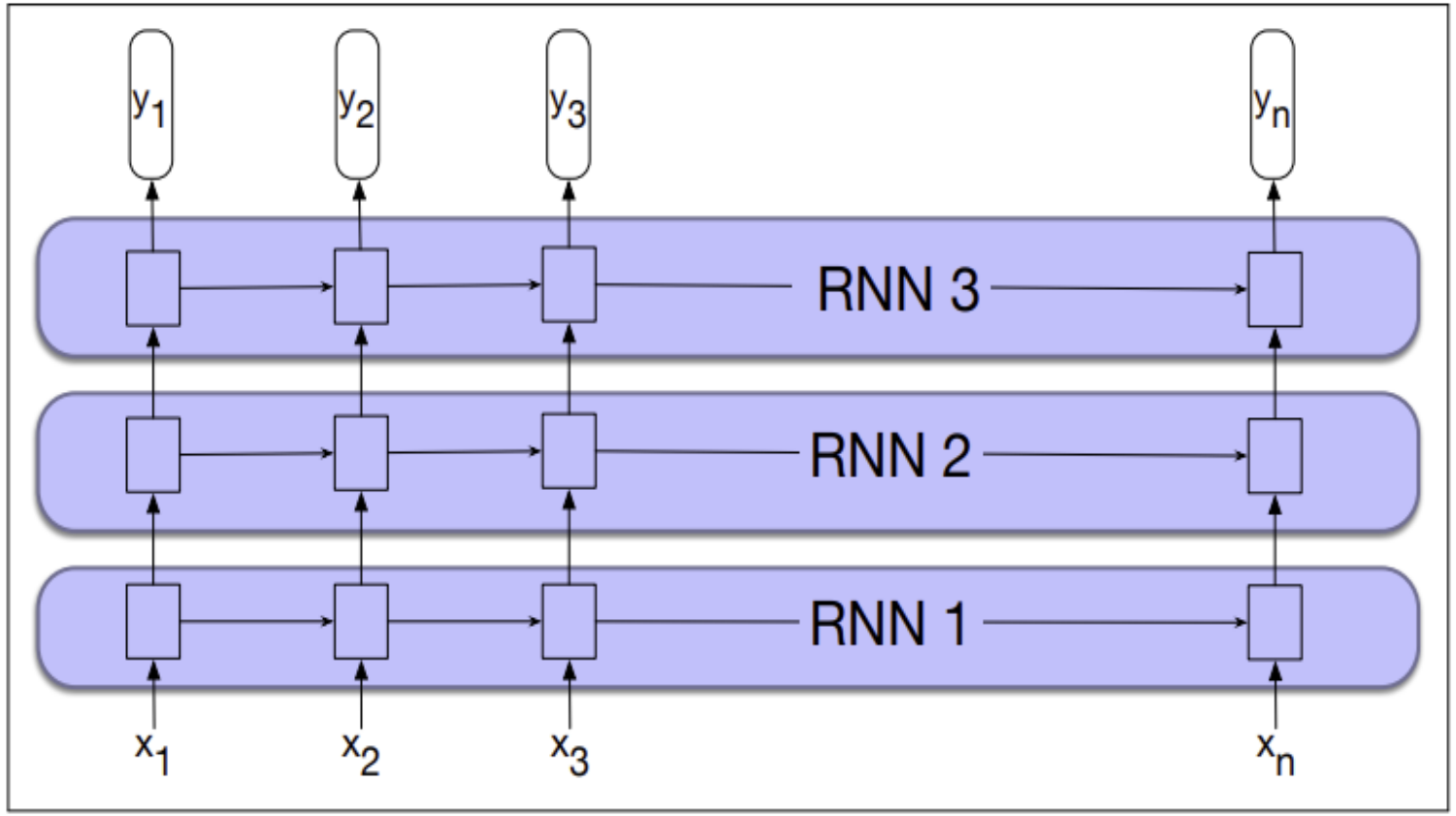


Figure 9.10 Stacked recurrent networks. The output of a lower level serves as the input to higher levels with the output of the last network serving as the final output.

Stacked RNNs

- Stacked RNNs generally outperform single-layer networks. One reason for this success seems to be that the network induces representations at differing levels of abstraction across layers. Just as the early stages of the human visual system detect edges that are then used for finding larger regions and shapes, the initial layers of stacked networks can induce representations that serve as useful abstractions for further layers—representations that might prove difficult to induce in a single RNN. The optimal number of stacked RNNs is specific to each application and to each training set. However, as the number of stacks is increased the training costs rise quickly.

Bidirectional RNNs

- The RNN uses information from the left (prior) context to make its predictions at time t. But in many applications we have access to the entire input sequence; in those cases we would like to use words from the context to the right of t. One way to do this is to run two separate RNNs, one left-to-right, and one right-to-left, and concatenate their representations.

Bidirectional RNNs

- In the left-to-right RNNs, the hidden state at a given time t represents everything the network knows about the sequence up to that point. The state is a function of the inputs x1,..., xt and represents the context of the network to the left of the current time.

$$\mathbf{h}_t^f = \text{RNN}_{\text{forward}}(\mathbf{x}_1, \ldots, \mathbf{x}_t) \qquad (9.16)$$

This new notation $\mathbf{h}_t^f$ simply corresponds to the normal hidden state at time t, representing everything the network has gleaned from the sequence so far. To take advantage of context to the right of the current input, we can train an RNN on a reversed input sequence. With this approach, the hidden state at time t represents information about the sequence to the right of the current input:

Bidirectional RNNs

$$\mathbf{h}_t^b = \text{RNN}_{\text{backward}}(\mathbf{x}_t, \ldots \mathbf{x}_n) \qquad (9.17)$$

Here, the hidden state $\mathbf{h}_t^b$ represents all the information we have discerned about the sequence from t to the end of the sequence

A bidirectional RNN (Schuster and Paliwal, 1997) combines two independent bidirectional RNN RNNs, one where the input is processed from the start to the end, and the other from the end to the start. We then concatenate the two representations computed by the networks into a single vector that captures both the left and right contexts of an input at each point in time

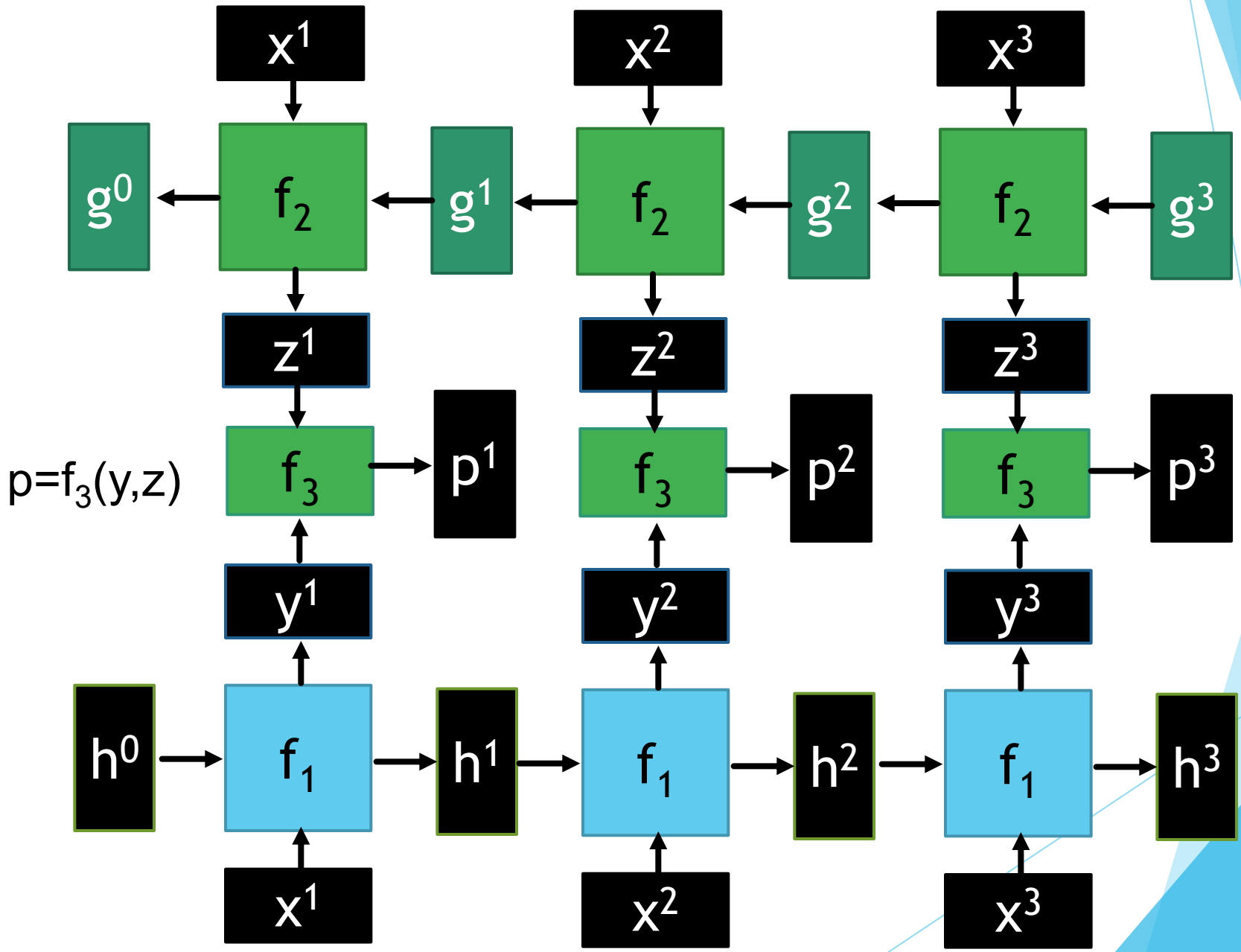
Bidirectional RNNs

- Here we use either the semicolon ";" or the equivalent symbol $\oplus$ to mean vector concatenation:

$$\begin{aligned} \mathbf{h}_t &= [\mathbf{h}_t^f ; \mathbf{h}_t^b] \\ &= \mathbf{h}_t^f \oplus \mathbf{h}_t^b \end{aligned} \tag{9.18}$$

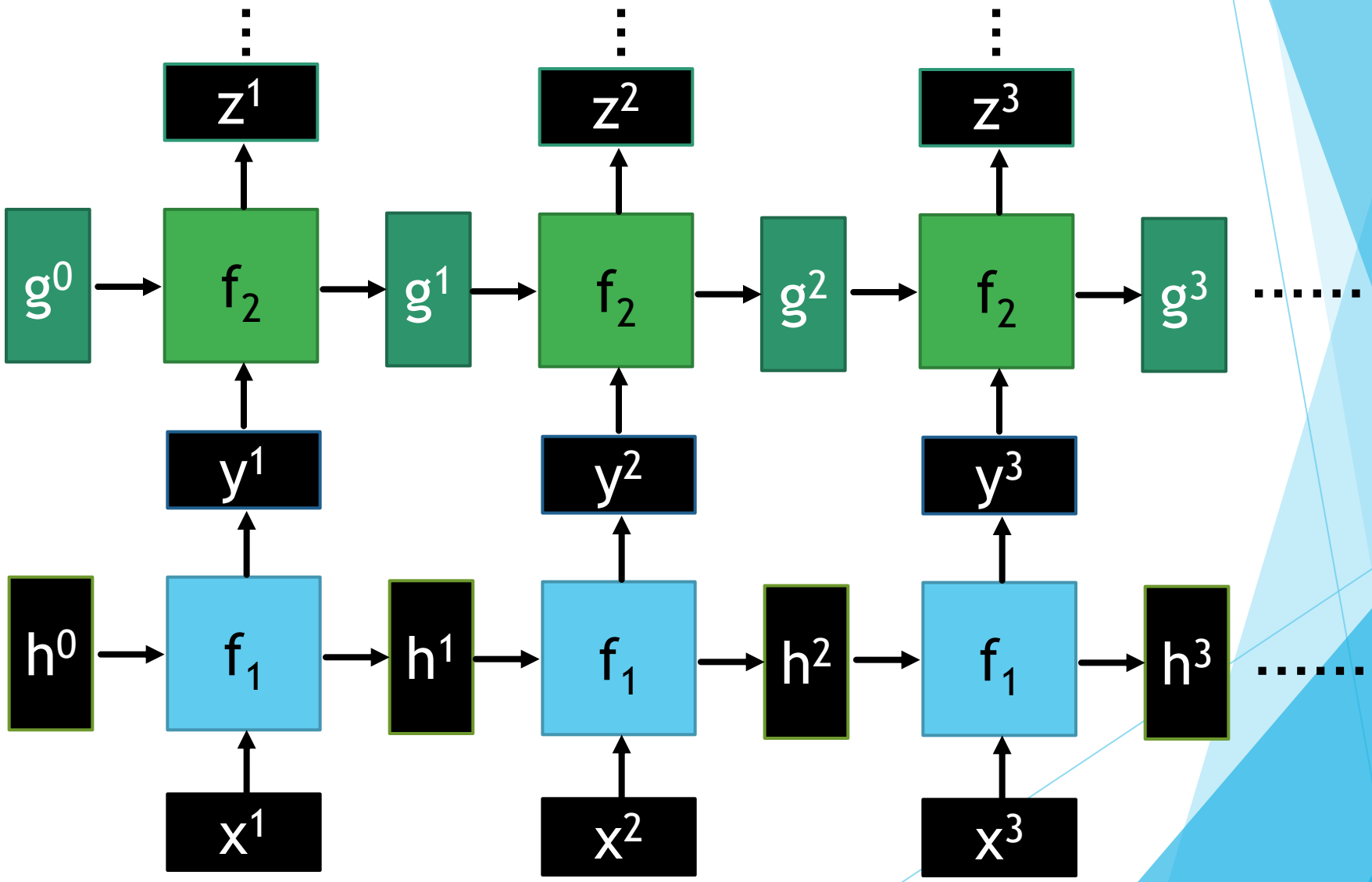
Bidirectional RNN

$y,h=f_1(x,h)$ $z,g=f_2(g,x)$



Deep RNN

$h', y = f_1(h, x)$, $g', z = f_2(g, y)$ ...

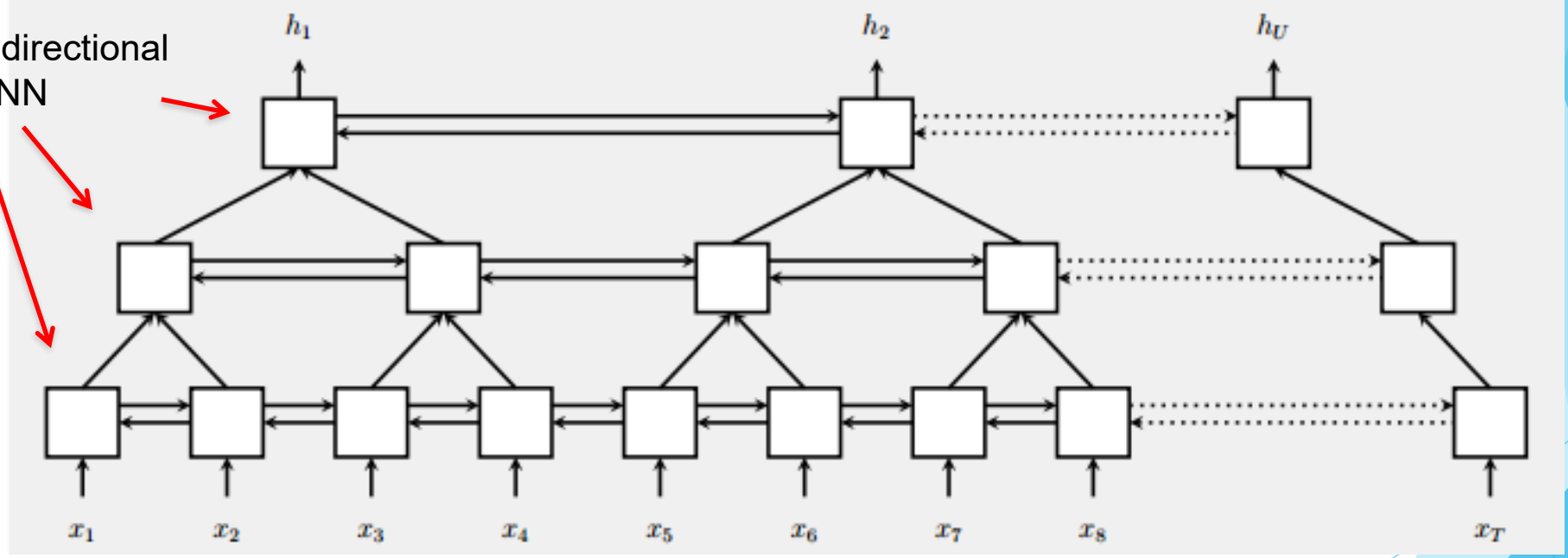


Pyramid RNN

Significantly speed up training

- Reducing the number of time steps

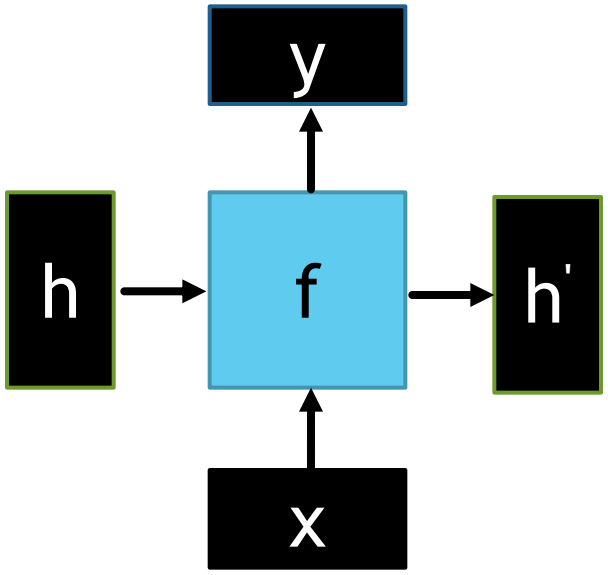
Bidirectional RNN



W. Chan, N. Jaitly, Q. Le and O. Vinyals, "Listen, attend and spell: A neural network for large vocabulary conversational speech recognition," ICASSP, 2016

Naïve RNN

- Given function f: h',y=f(h,x)



$$h' = \sigma(W^h h + W^i x)$$

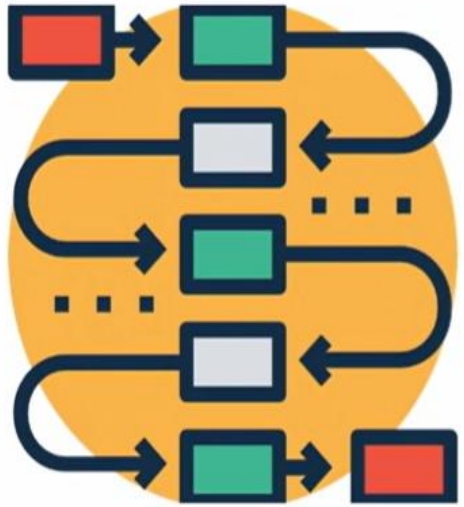
$$y = \sigma(W^o h')$$

softmax

Note, y is computed from h'

We have ignored the bias

Problems with RNN



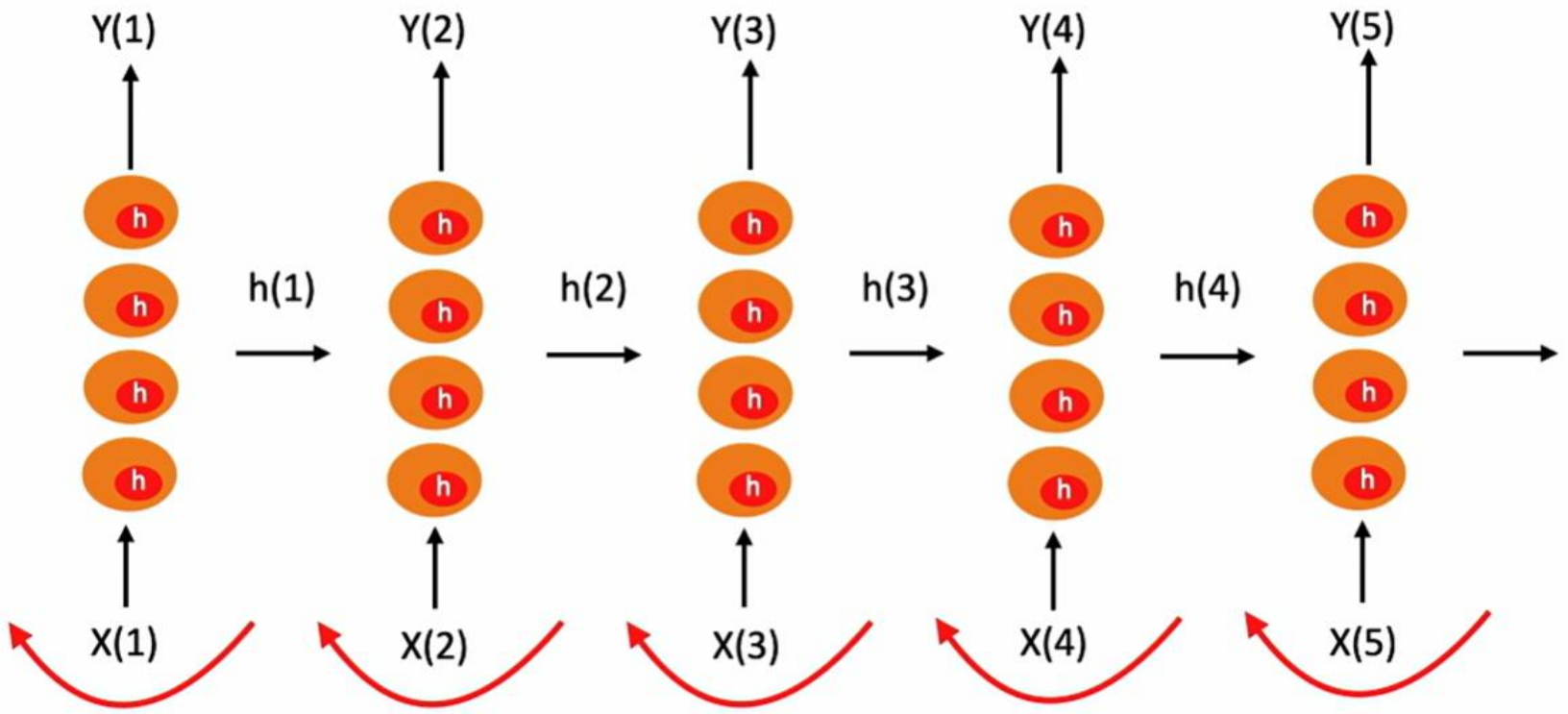
Recurrent cells are capable of remembering the past

However the memory capacity of regular recurrent neurons is not very large

If you have long input sequences your recurrent neurons may not process these well

Neurons will not be able to remember information from time instances in the distant past

Layers of RNN



Number of layers = Number of time instances in the input data

Gradient propagated back through the unrolled layer of the neural network

Problems with RNN

- When dealing with a time series, it tends to forget old information. When there is a distant relationship of unknown length, we wish to have a “memory” to it. Vanishing gradient problem.

Propagating gradients back through many layers may cause the gradients to become very small or explode in magnitude

RNNs are prone to the vanishing and exploding gradients - leads to poor performance, long training periods

Problems with RNN

- Vanishing gradient problem is a phenomenon that occurs during the training of deep neural networks, where the gradients that are used to update the network become extremely small or "vanish" as they are backpropogated from the output layers to the earlier layers.
- During the training process of the neural network, the goal is to minimize a loss function by adjusting the weights of the network.
- The backpropogation algorithm calculates these gradients by propogating the error from the output layer to the input layer.

Problems with RNN

- The vanishing gradient problem causes the gradients to shrink. But, if a gradient is small, it won't be possible to effectively update the weights and biases of the initial layers with each training session.

- These initial layers are vital for recognizing the core elements of the input data, so, if their weights and biases are not properly updated, it is possible that the entire network could be inaccurate.

Problems with RNN

- One reason for the inability of RNNs to carry forward critical information is that the hidden layers, and, by extension, the weights that determine the values in the hidden layer, are being asked to perform two tasks simultaneously: provide information useful for the current decision, and updating and carrying forward information required for future decisions.

- A second difficulty with training RNNs arises from the need to backpropagate the error signal back through time. A frequent result of this process is that the gradients are eventually driven to zero which causes the vanishing gradients problem.

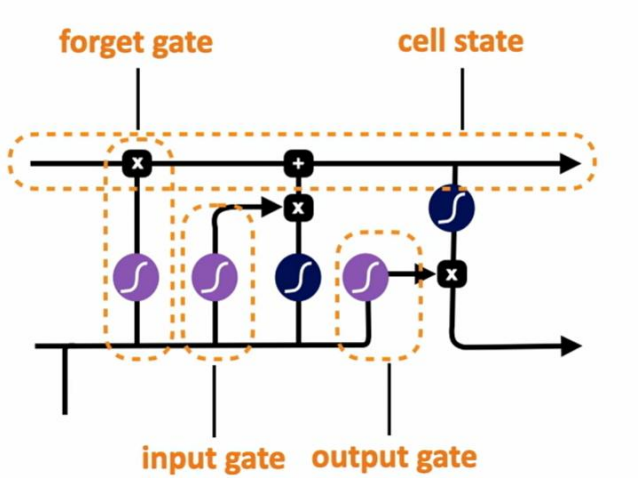
LSTM

- To address these issues, more complex network architectures have been designed to explicitly manage the task of maintaining relevant context over time, by enabling the network to learn to forget information that is no longer needed and to remember information required for decisions still to come.

- The most commonly used such extension to RNNs is the long short-term memory (LSTM) (Hochreiter and Schmidhuber, 1997) text management problem into two subproblems: removing information no longer needed from the context, and adding information likely to be needed for later decision making. The key to solving both problems is to learn how to manage this context rather than hard-coding a strategy into the architecture. LSTMs accomplish this by first adding an explicit context layer to the architecture, and through the use of specialized neural units that make use of gates to control the flow of information into and out of the units that comprise the network layers. These gates are implemented through the use of additional weights that operate sequentially on the input, and previous hidden layer, and previous context layers

LSTM

- The gates in an LSTM share a common design pattern; each consists of a feedforward layer, followed by a sigmoid activation function, followed by a pointwise multiplication with the layer being gated. The choice of the sigmoid as the activation function arises from its tendency to push its outputs to either 0 or 1. Combining this with a pointwise multiplication has an effect similar to that of a binary mask. Values in the layer being gated that align with values near 1 in the mask are passed through nearly unchanged; values corresponding to lower values are essentially erased.



LSTM

- Forget gate:
- The purpose of this gate is to delete information from the context that is no longer needed. The forget gate computes a weighted sum of the previous state's hidden layer and the current input and passes that through a sigmoid. This mask is then multiplied element-wise by the context vector to remove the information from context that is no longer required.

$$\mathbf{f}_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t) \quad (9.20)$$

$$\mathbf{k}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t \quad (9.21)$$

LSTM

- The next task is to compute the actual information we need to extract from the previous hidden state and current inputs—the same basic computation we've been using for all our recurrent networks.

$$\mathbf{g}_t = \tanh(\mathbf{U}_g\mathbf{h}_{t-1} + \mathbf{W}_g\mathbf{x}_t) \quad (9.22)$$

Add get: Next, we generate the mask for the add gate to select the information to add to the current context

$$\mathbf{i}_t = \sigma(\mathbf{U}_i\mathbf{h}_{t-1} + \mathbf{W}_i\mathbf{x}_t) \quad (9.23)$$

$$\mathbf{j}_t = \mathbf{g}_t \odot \mathbf{i}_t \quad (9.24)$$

LSTM

- Next, we add this to the modified context vector to get our new context vector.

$$\mathbf{c}_t = \mathbf{j}_t + \mathbf{k}_t \tag{9.25}$$

Output gate: The final gate we'll use is the output gate which is used to decide what information is required for the current hidden state (as opposed to what information needs to be preserved for future decisions).

$$\mathbf{o}_t = \sigma(\mathbf{U}_o\mathbf{h}_{t-1} + \mathbf{W}_o\mathbf{x}_t) \tag{9.26}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \tag{9.27}$$

LSTM

- Fig. 9.13 an LSTM accepts as input the context layer, and hidden layer from the previous time step, along with the current input vector. It then generates updated context and hidden vectors as output. It is the hidden state, ht , that provides the output for the LSTM at each time step. This output can be used as the input to subsequent layers in a stacked RNN, or at the final layer of a network ht can be used to provide the final output of the LSTM.

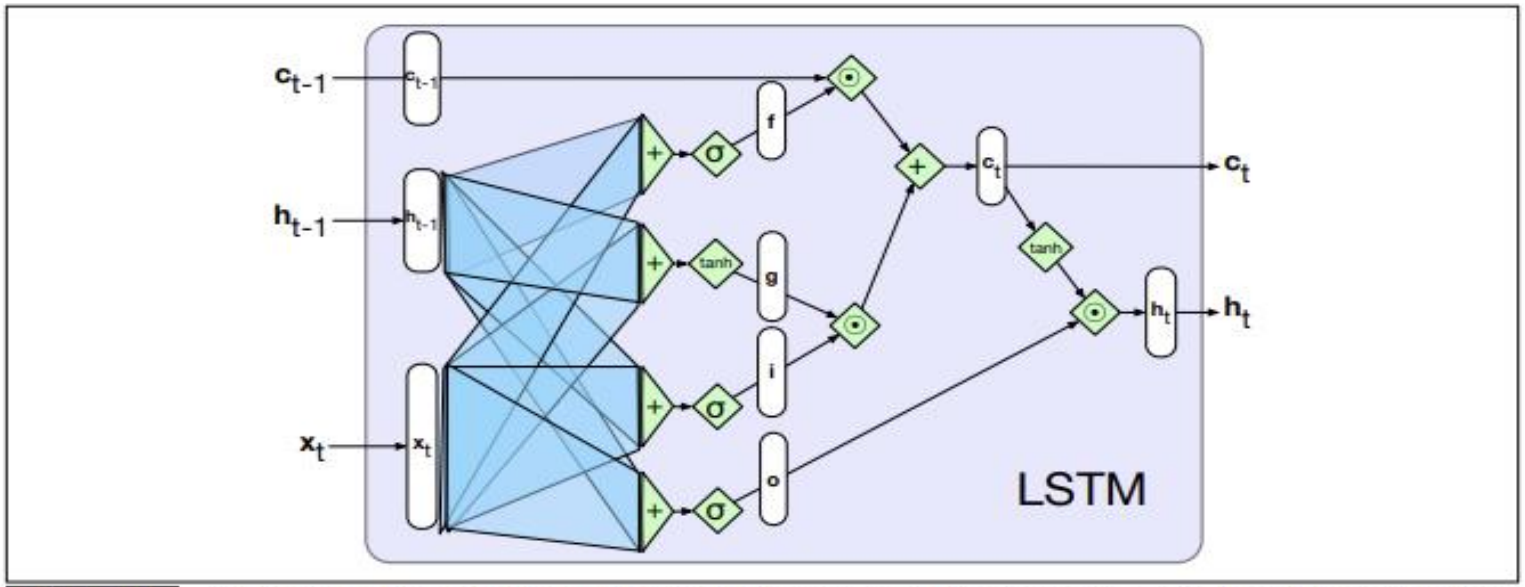


Figure 9.13 A single LSTM unit displayed as a computation graph. The inputs to each unit consists of the current input, x, the previous hidden state, h_{t-1}, and the previous context, c_{t-1}. The outputs are a new hidden state, h_t and an updated context, c_t.

LSTM

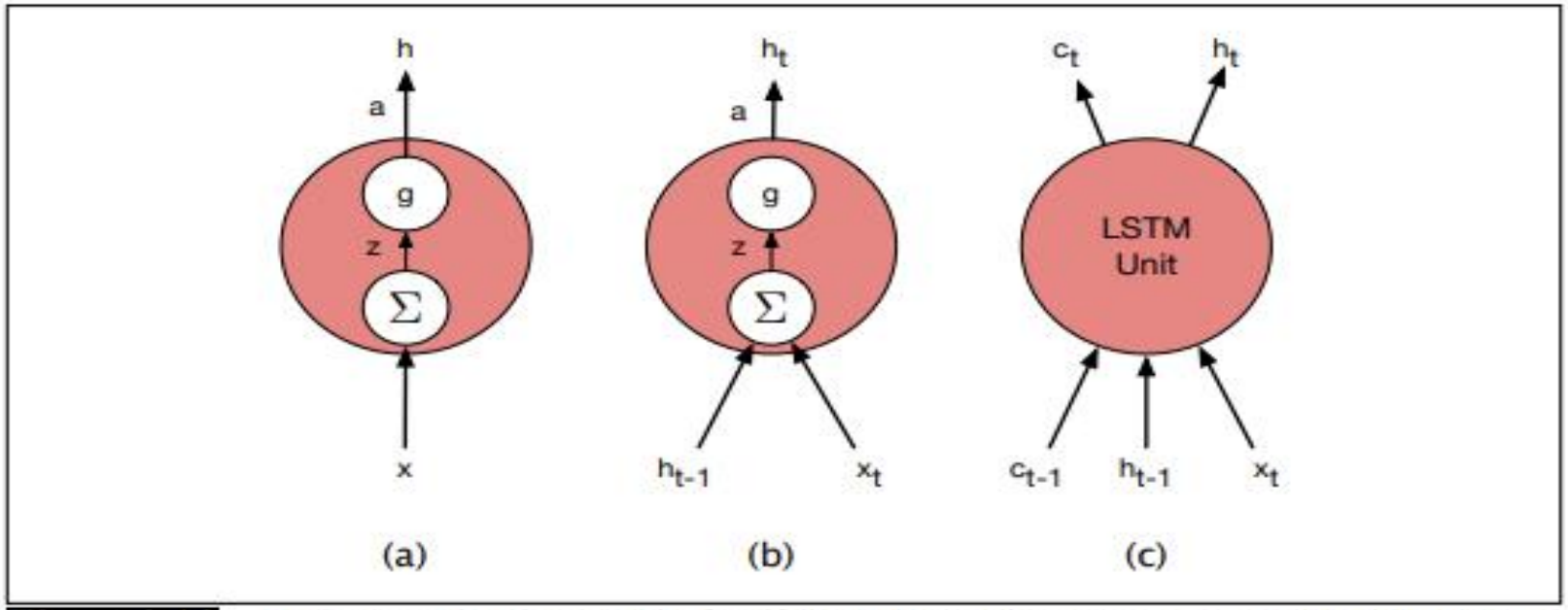
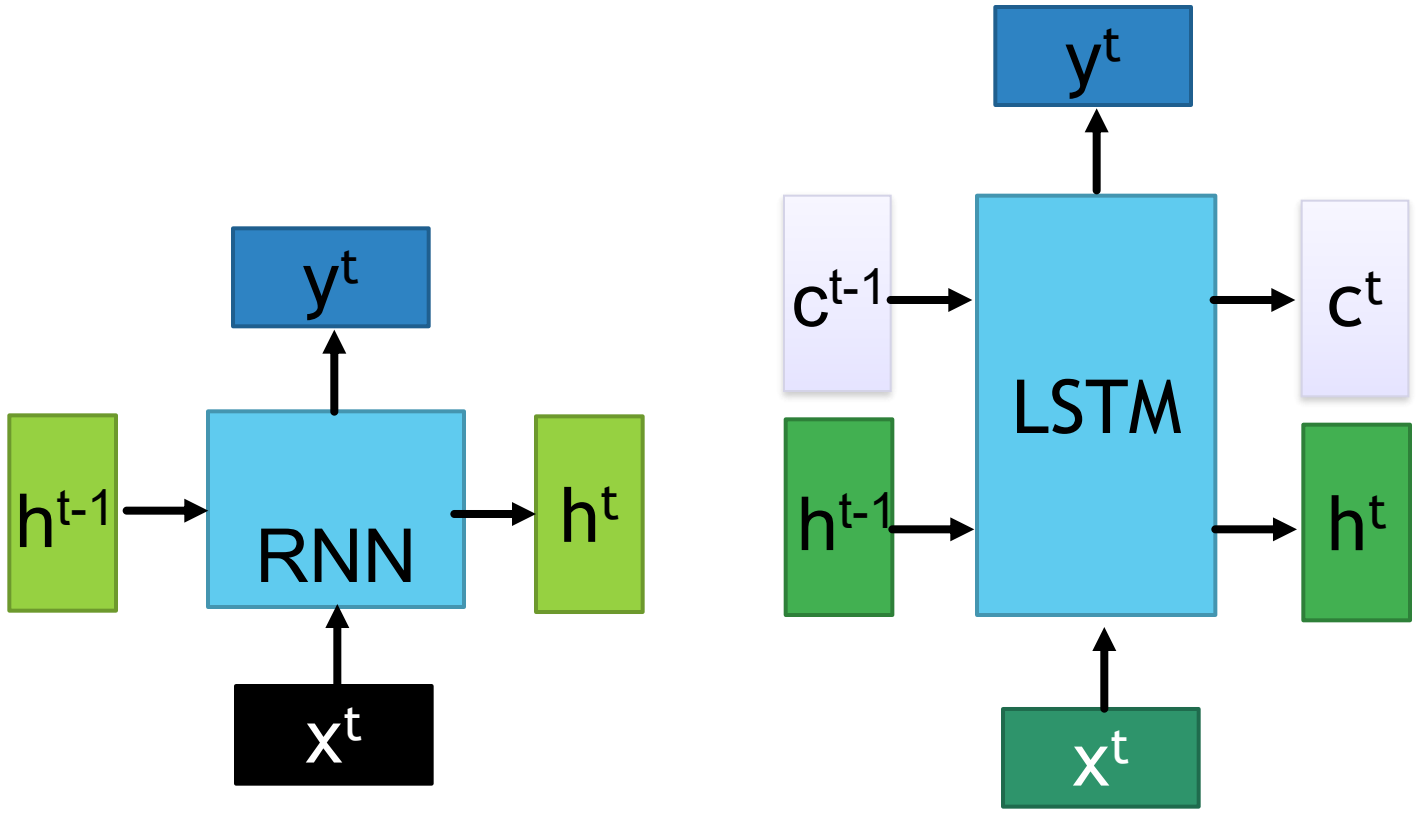
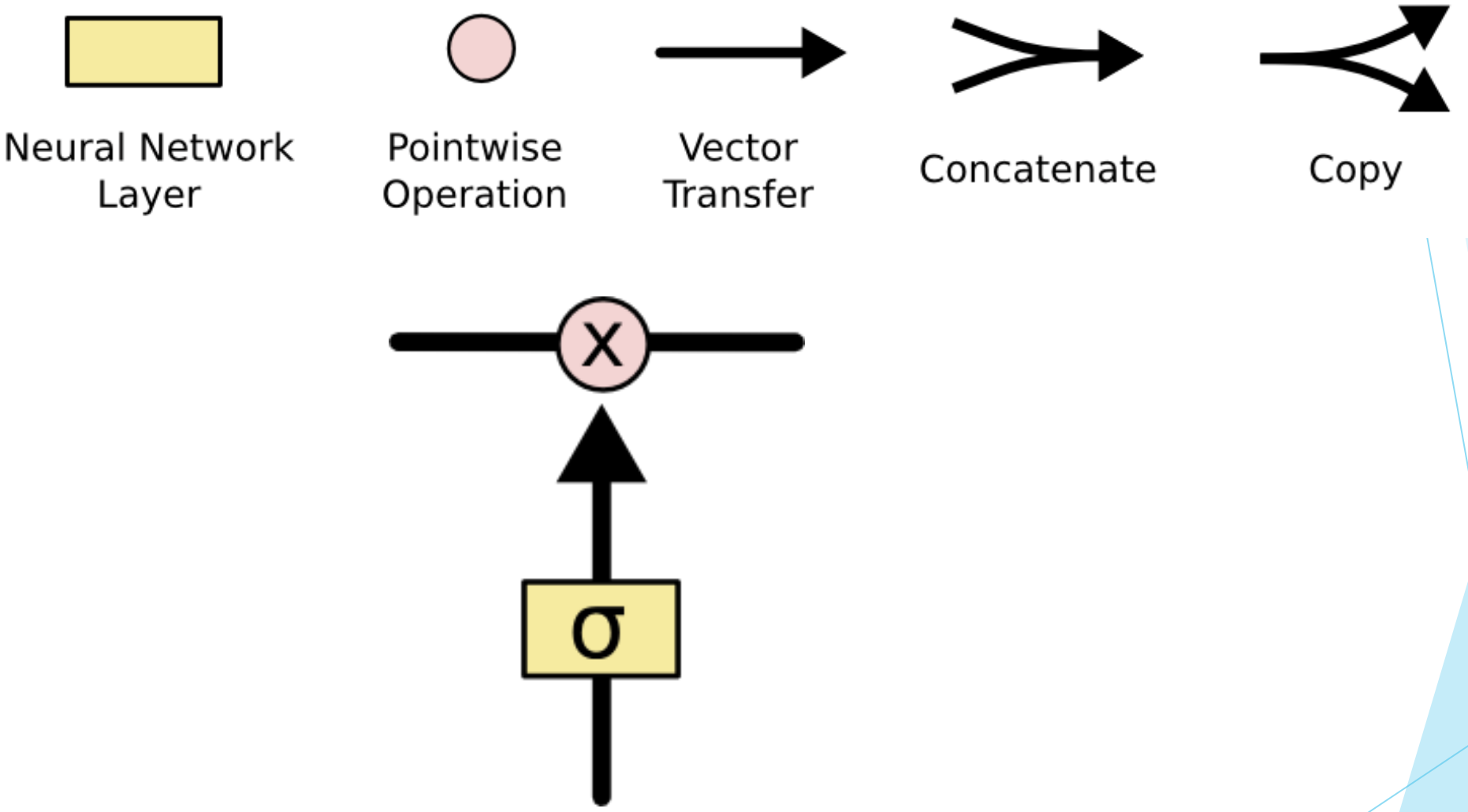


Figure 9.14 Basic neural units used in feedforward, simple recurrent networks (SRN), and long short-term memory (LSTM).

RNN vs LSTM

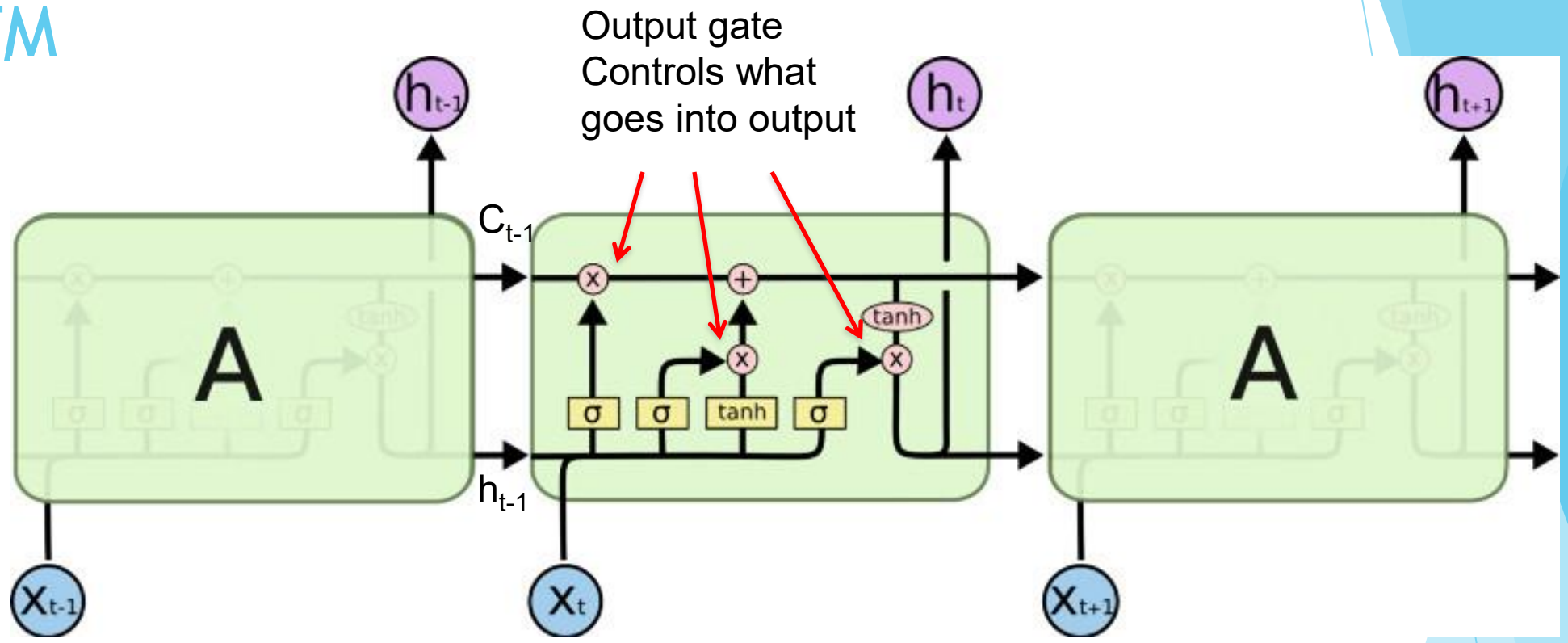


LSTMs offer improvements over traditional RNNs by addressing the vanishing gradient problem and providing a more effective mechanism for handling long-term dependencies. LSTMs are better suited for tasks requiring memory over long sequences, making them a popular choice for various applications in natural language processing, time-series analysis, and beyond.

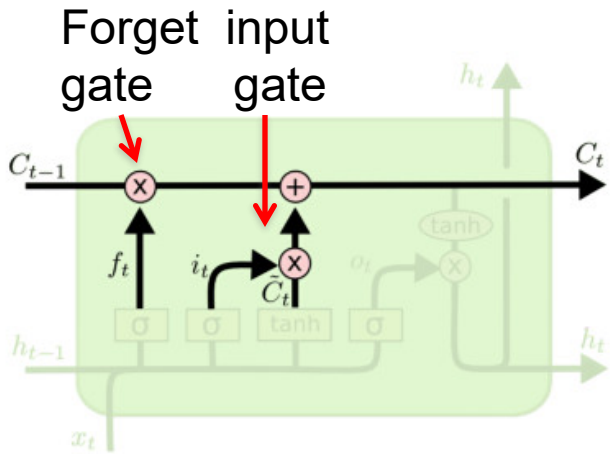


The sigmoid layer outputs numbers between 0-1 determine how much each component should be let through. Pink X gate is point-wise multiplication.

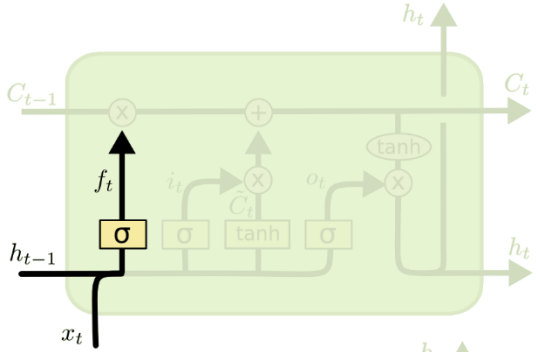
LSTM



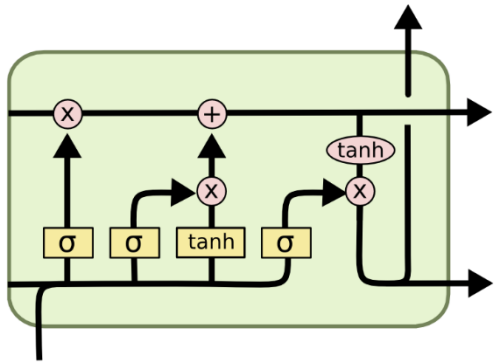
The core idea is this cell state C_t, it is changed slowly, with only minor linear interactions. It is very easy for information to flow along it unchanged.

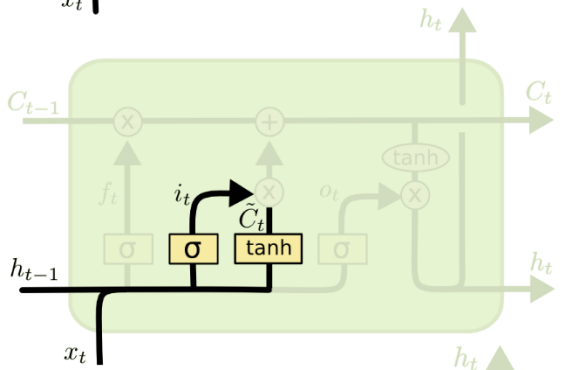


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



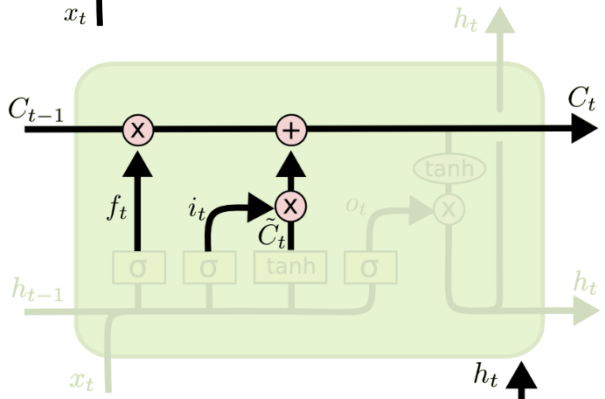
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$





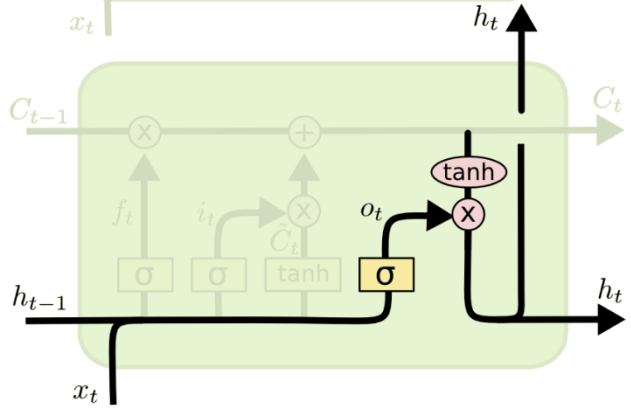
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

i_t decides what component is to be updated.
C'_t provides change contents



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

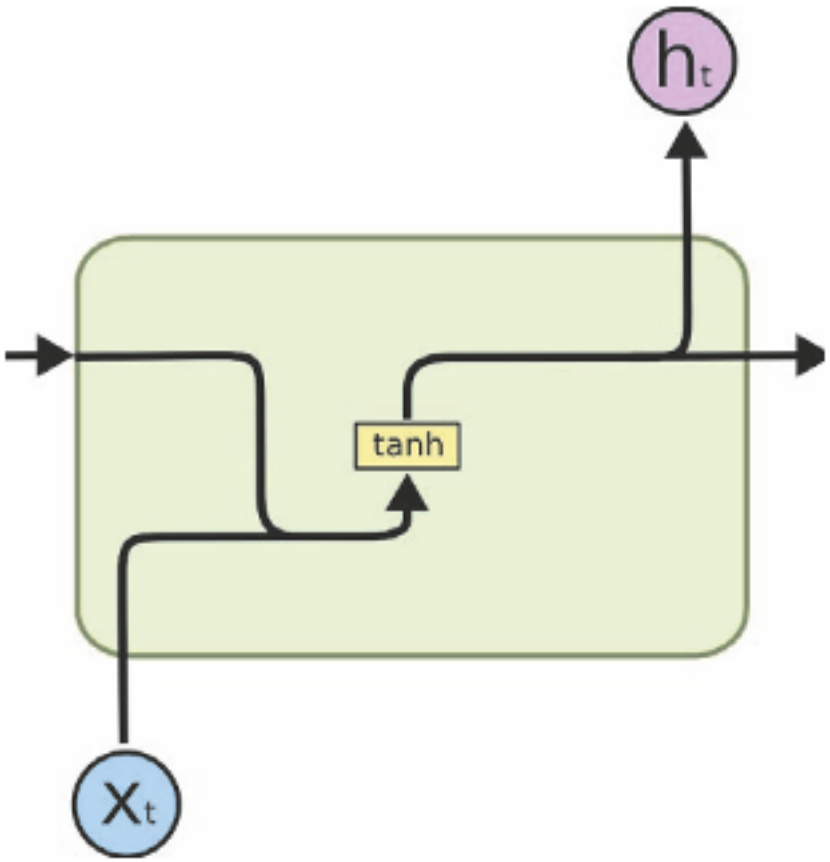
Updating the cell state



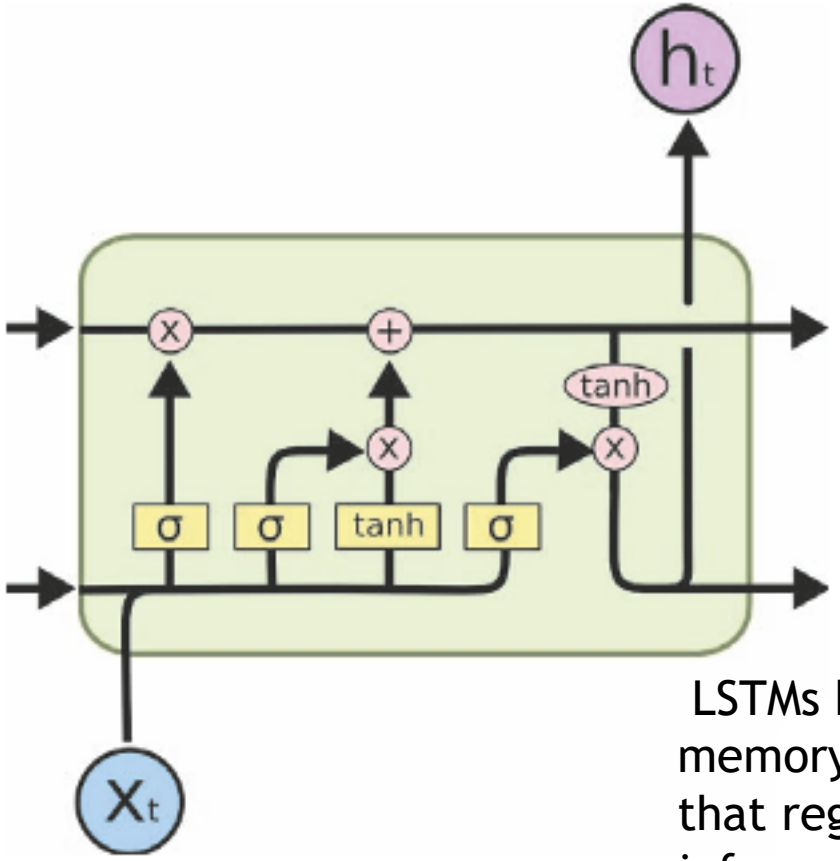
$$o_t = \sigma\left(W_o \; [h_{t-1}, x_t] \; + \; b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$

Decide what part of the cell state to output

RNN vs LSTM



(a) RNN

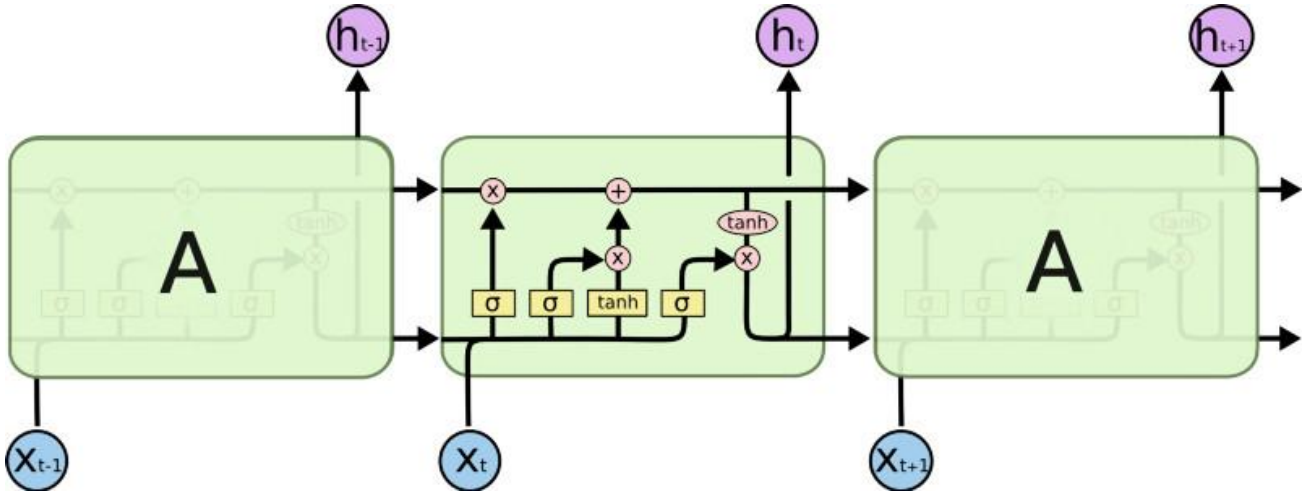


(b) LSTM

LSTMs were specifically designed to address the vanishing gradient problem in RNNs and handle long-term dependencies more effectively.

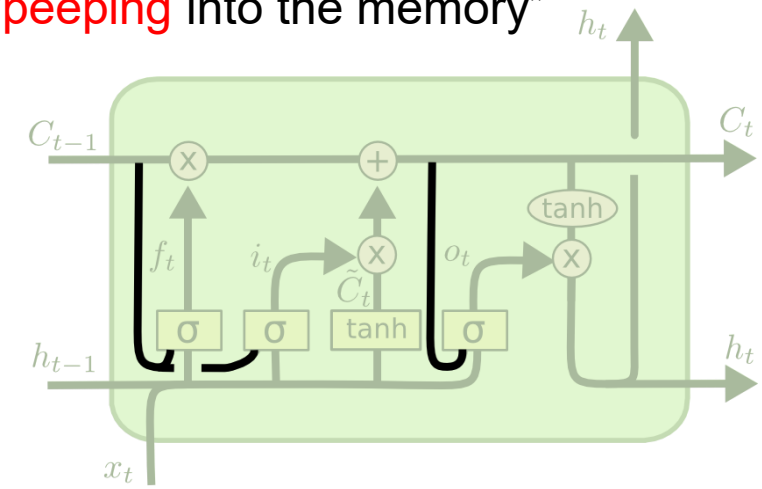
LSTMs have a more sophisticated memory mechanism with gated cells that regulate the flow of information. The cell state in LSTMs allows them to store and update information over time, making them better at capturing long-term dependencies

Peephole LSTM

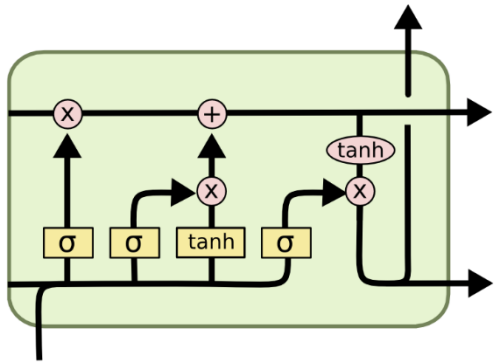


Gers and Schmidhuber have connected LSTM memory cells with peepholes from cell to gates. Also, peephole connections allow gates to use the previous internal and hidden states. This allows peephole LSTM to learn more precise timings than traditional LSTM cells

Allows "peeping into the memory"



$$f_t = \sigma\left(W_f \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_f\right)$$
$$i_t = \sigma\left(W_i \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_i\right)$$
$$o_t = \sigma\left(W_o \cdot [\boldsymbol{C_t}, h_{t-1}, x_t] \; + \; b_o\right)$$



These 4 matrix computation should be done concurrently.

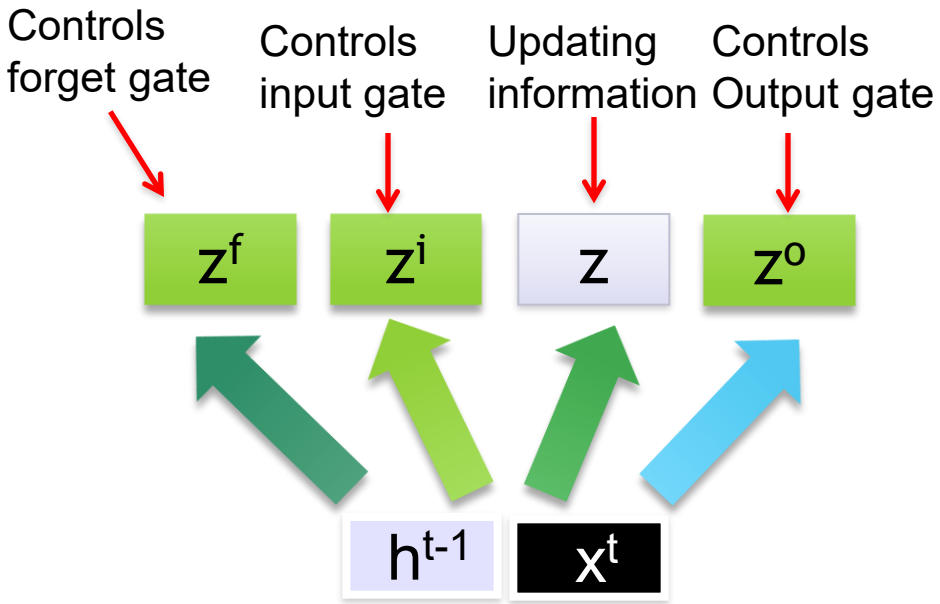
c^{t-1}

$$z = tanh\left(W \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix}\right)$$

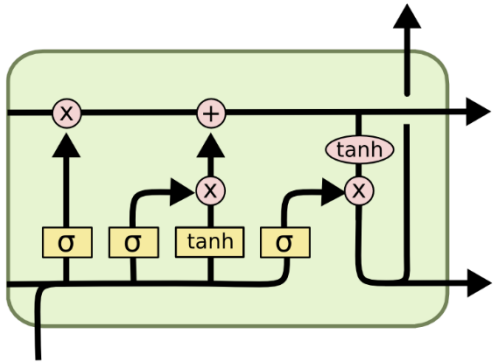
$$z^i = \sigma\left(W^i \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix}\right)$$

$$z_f = \sigma\left(W^f \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix}\right)$$

$$z^o = \sigma\left(W^o \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix}\right)$$



Information flow of LSTM



$z = \tanh\left(W \begin{bmatrix} x^t \\ h^{t-1} \\ c^{t-1} \end{bmatrix} \right)$

diagonal

c^{t-1}

"peephole"

z^o z^f z^i obtained by the same way

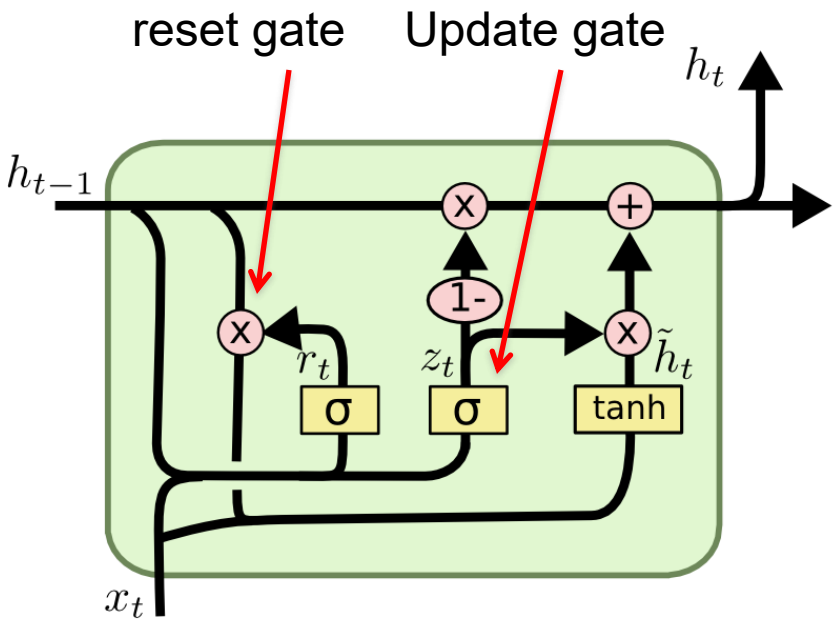
z^f z^i z z^o

h^{t-1} x^t

Information flow of LSTM

GRU – gated recurrent unit

(more compression)



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

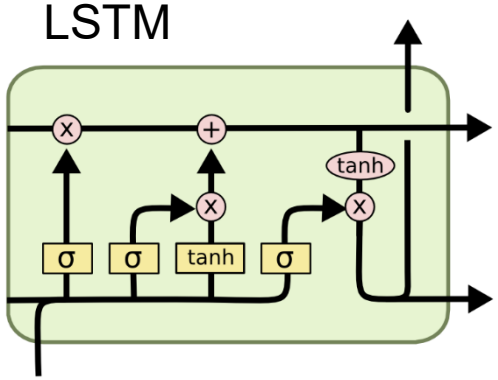
$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

It combines the forget and input into a single update gate.
It also merges the cell state and hidden state. This is simpler than LSTM. There are many other variants too.

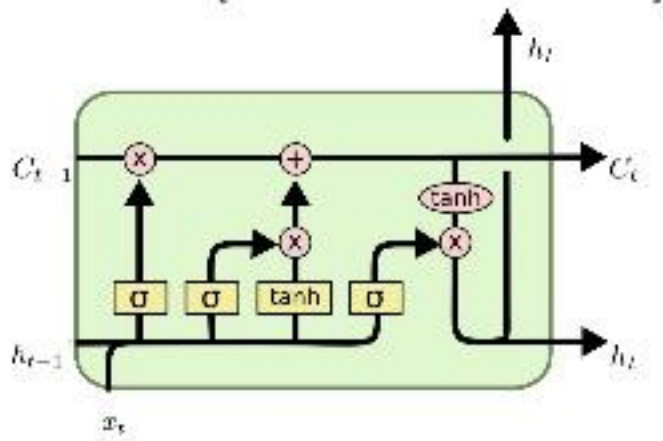
LSTM



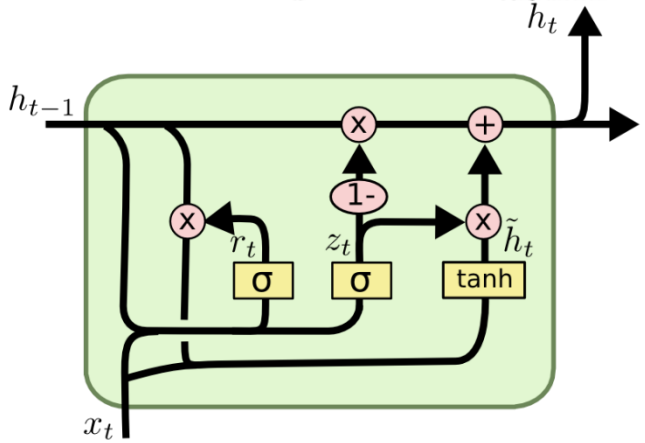
X,*: element-wise multiply

LSTM and GRU

- LSTM [Hochreiter&Schmidhuber97]

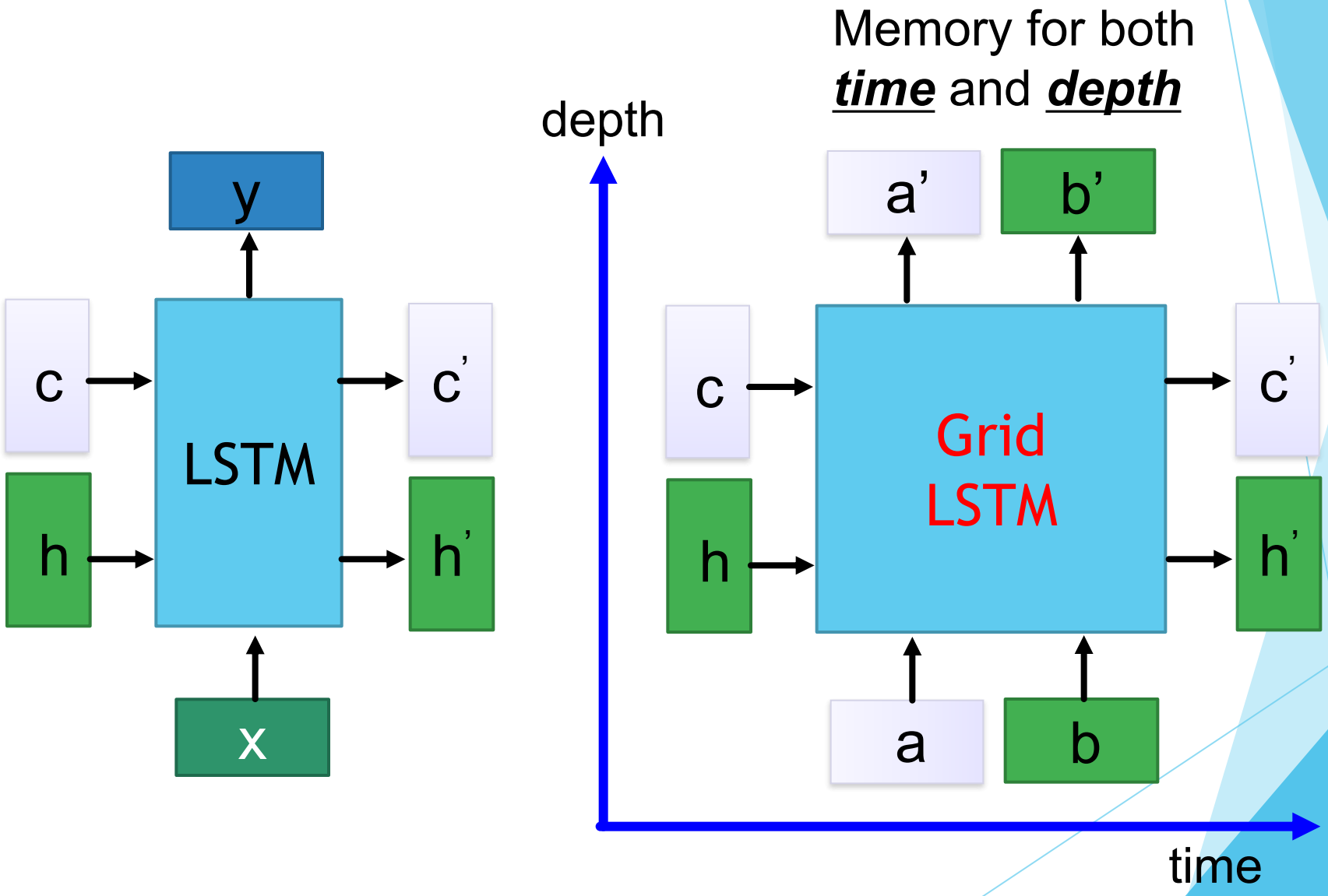


- GRU [Cho+14]

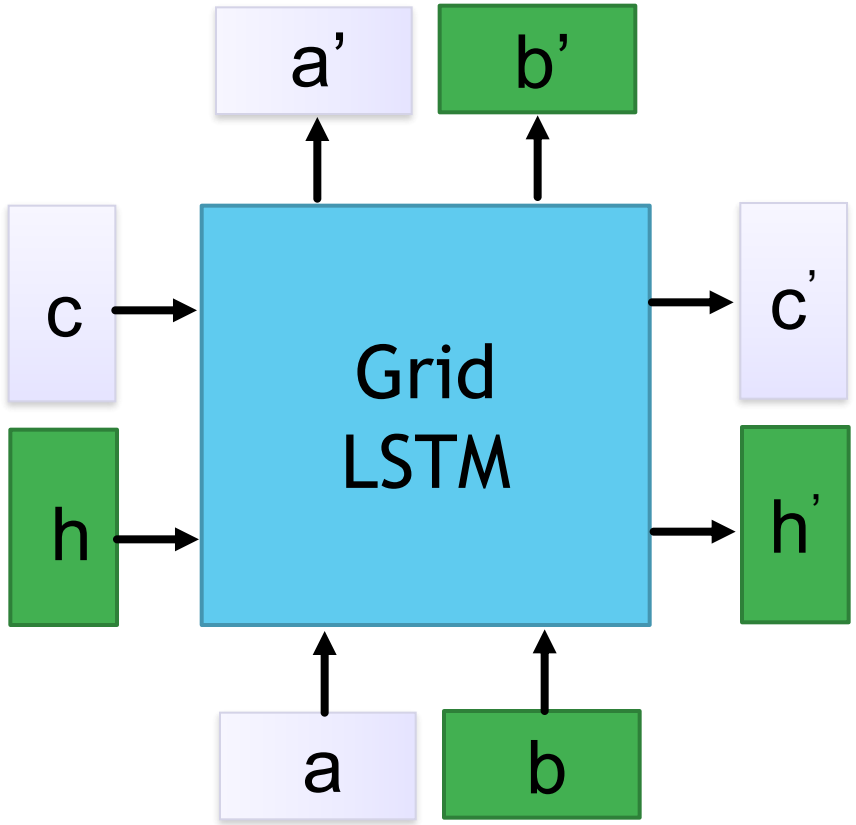


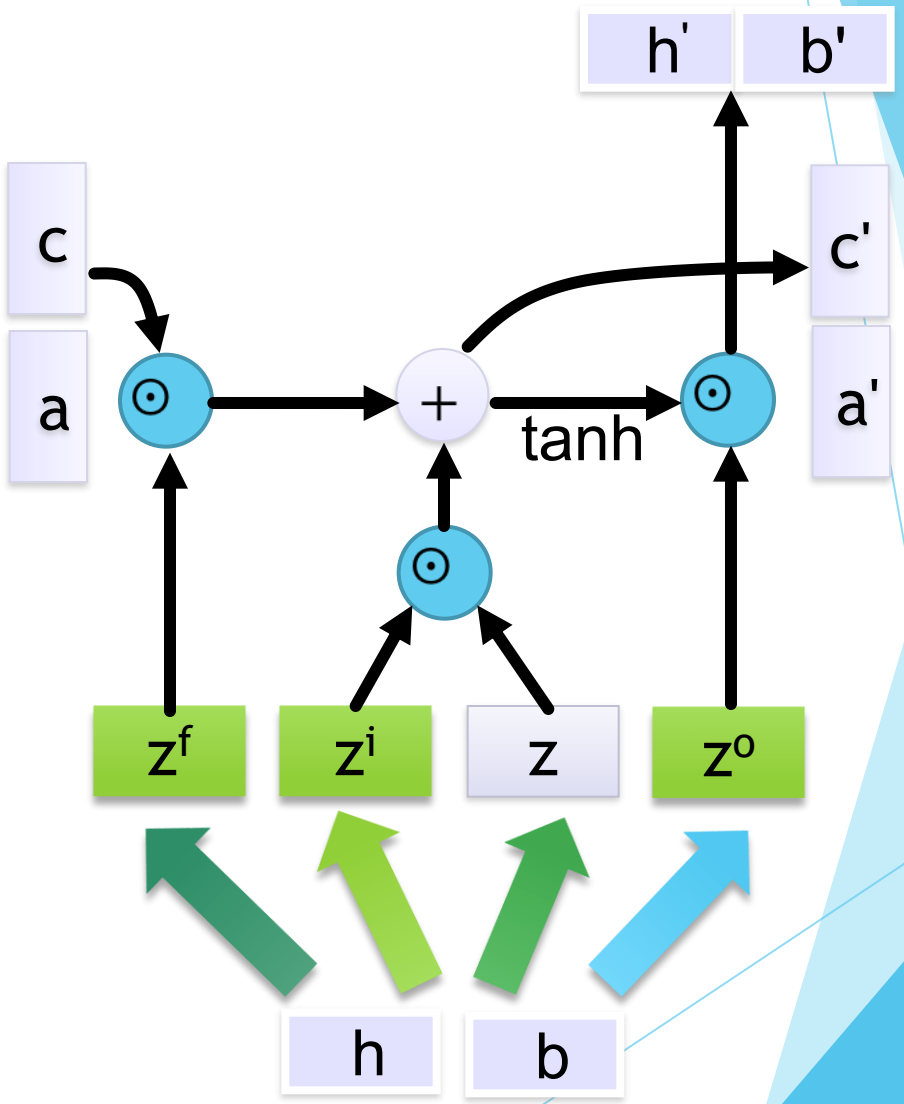
GRUs also takes x_t and h_{t-1} as inputs. They perform some calculations and then pass along h_t. What makes them different from LSTMs is that GRUs don't need the cell layer to pass values along. The calculations within each iteration ensure that the h_t values being passed along either retain a high amount of old information or are jump-started with a high amount of new information.

Grid LSTM


Memory for both
time and ***depth***
depth
y
c
LSTM
c'
h
h'
x
a'
b'
c
Grid
LSTM
c'
h
h'
a
b
time

Grid LSTM



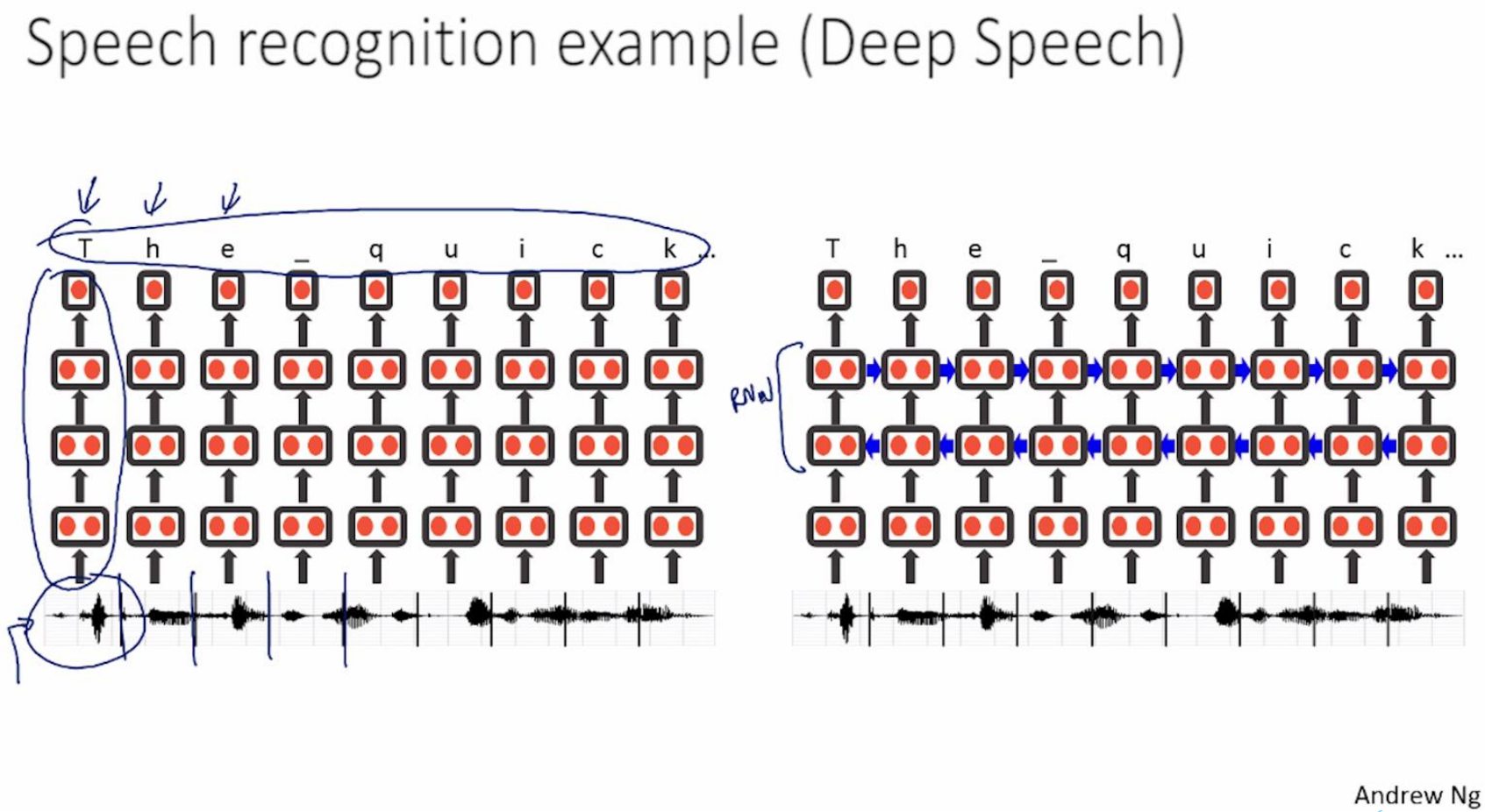


It can generalize to 3D, and more.

Applications of LSTM / RNN

- Neural machine translation
- Sequence to sequence chat model
- Chat with context

Baidu's speech recognition using RNN



Attention

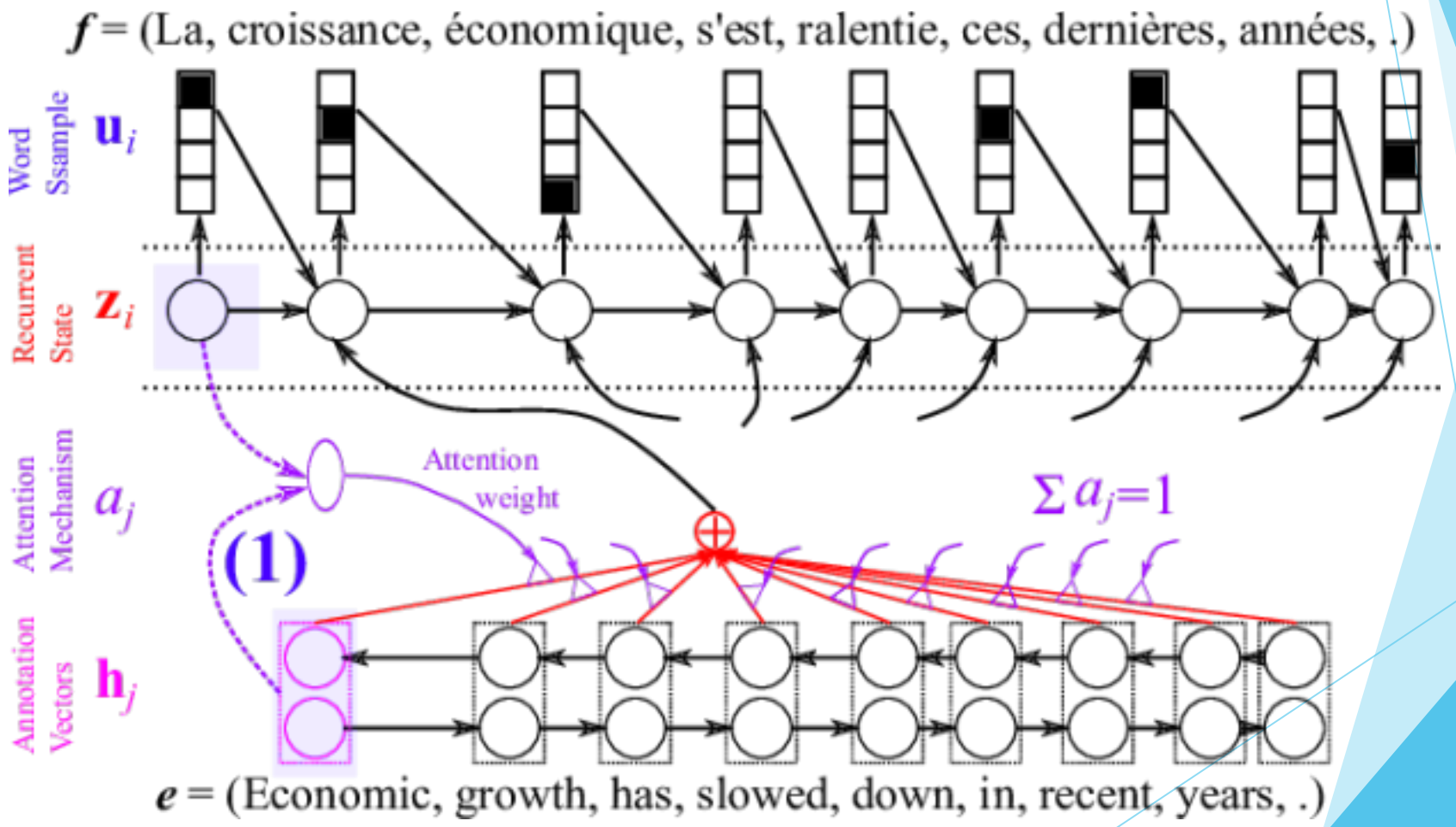

f = (La, croissance, économique, s'est, ralentie, ces, dernières, années, .)
Word Ssample
$\mathbf{u}_i$
Recurrent State
$\mathbf{z}_i$
Attention Mechanism
a_j
Attention weight
$\Sigma a_j=1$
(1)
Annotation Vectors
$\mathbf{h}_j$
e = (Economic, growth, has, slowed, down, in, recent, years, .)

Image caption generation using attention

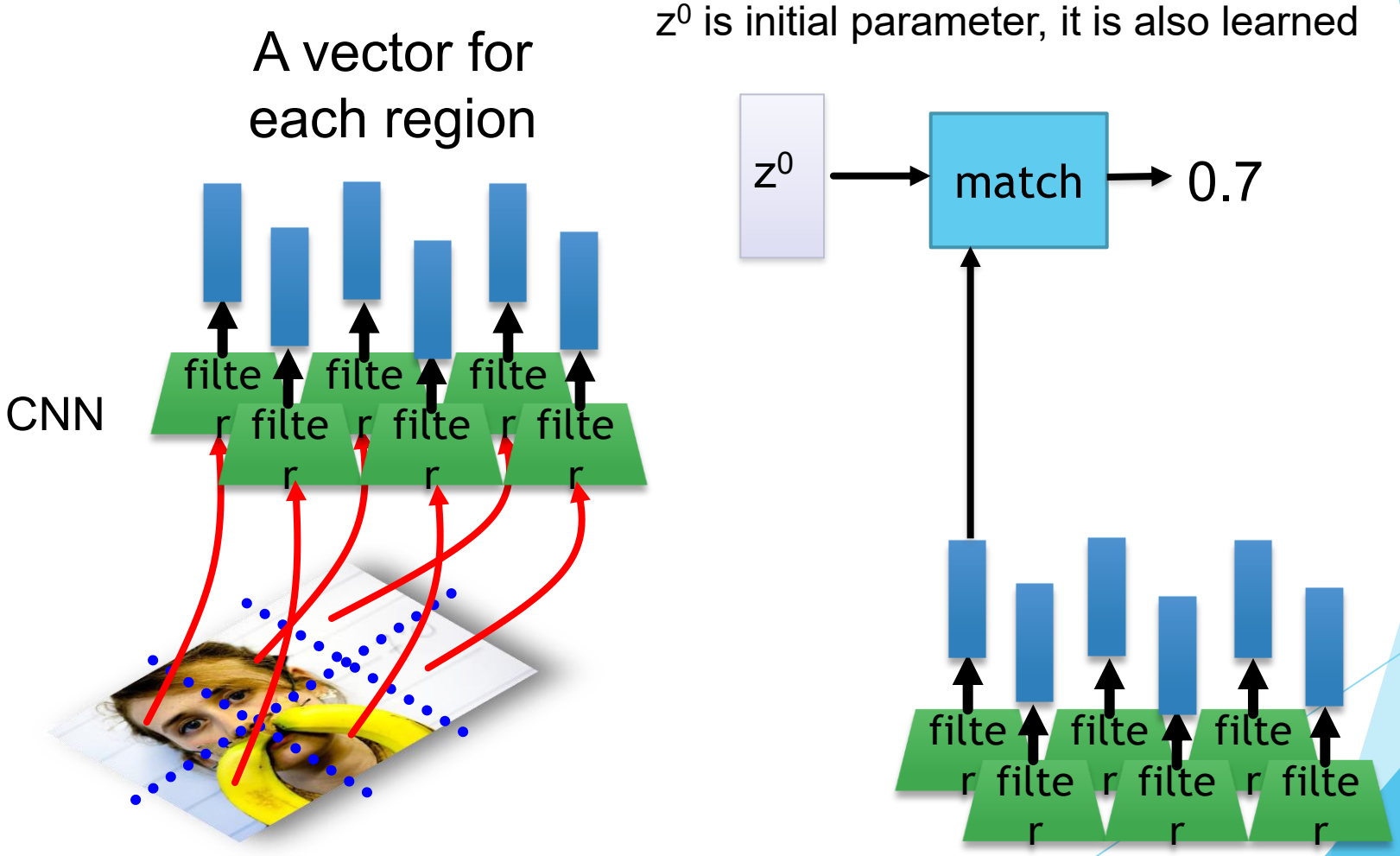


Image Caption Generation

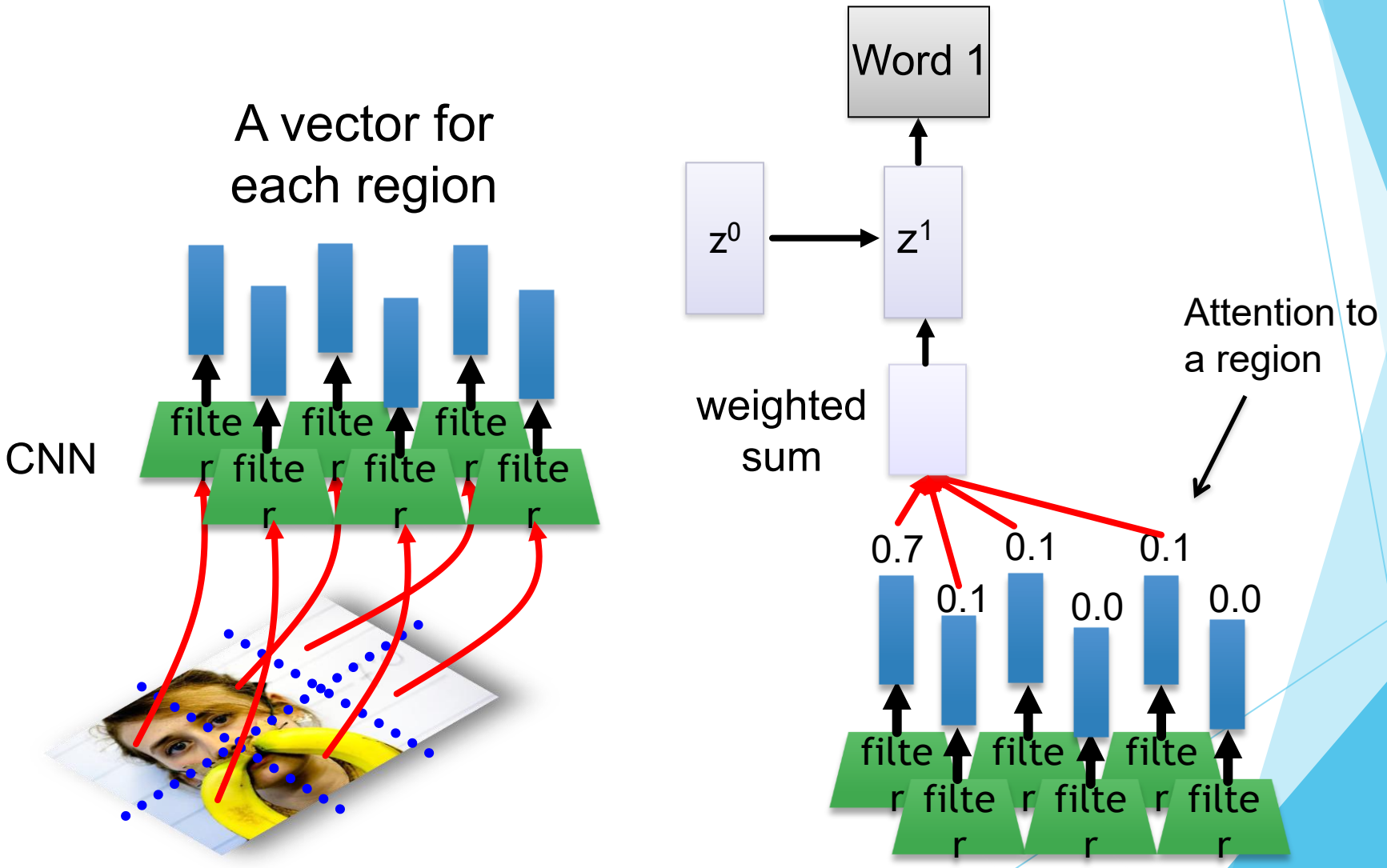


Image Caption Generation

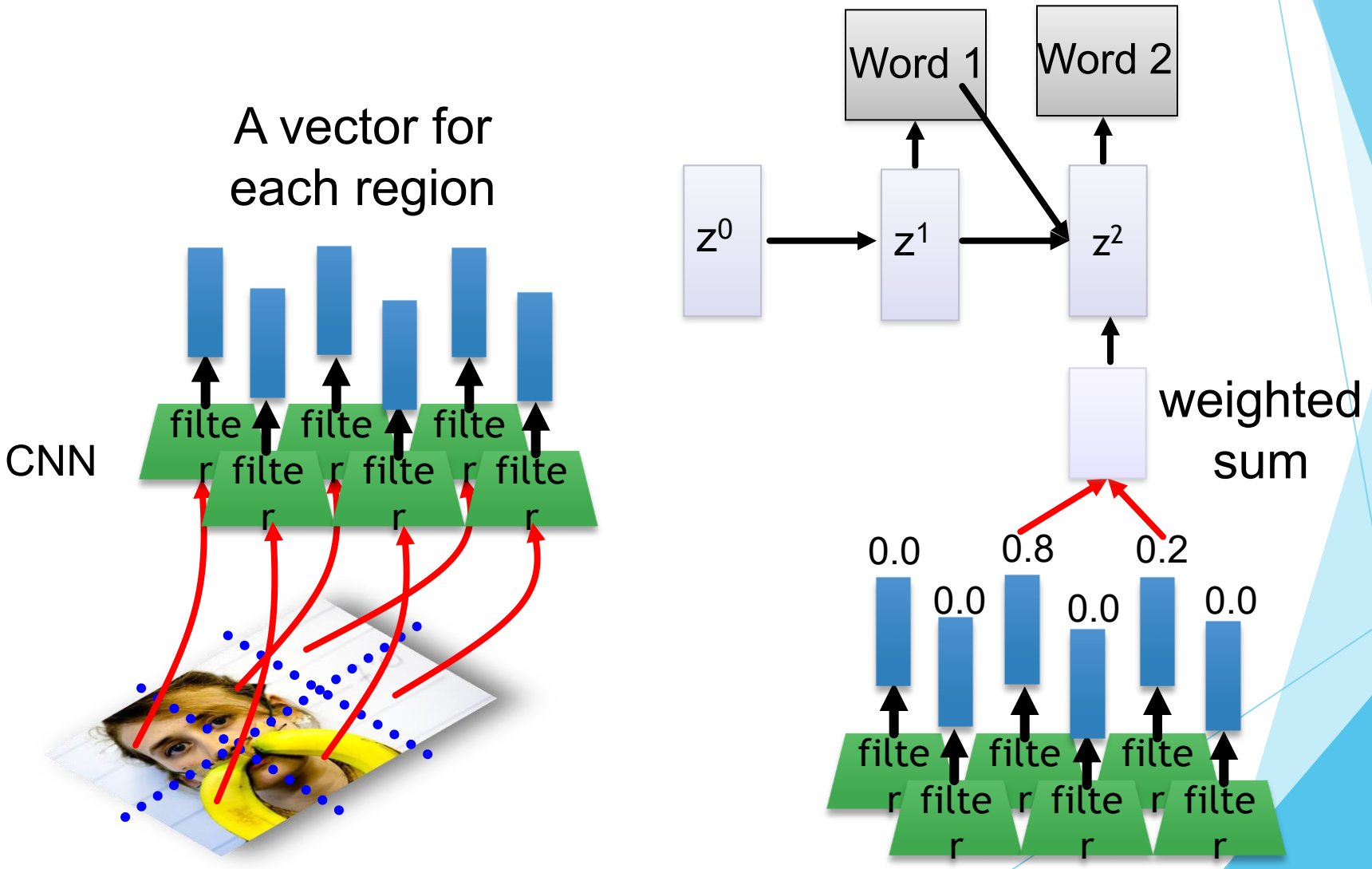


Image Caption Generation

A woman is throwing a frisbee in a park.

A dog is standing on a hardwood floor.

A stop sign is on a road with a mountain in the background.

A little girl sitting on a bed with a teddy bear.

A group of people sitting on a boat in the water.

A giraffe standing in a forest with trees in the background.

Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard Zemel, Yoshua Bengio, “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention”, ICML, 2015

Image Caption Generation

A large white bird standing in a forest.

A woman holding a clock in her hand.

A man wearing a hat and a hat on a skateboard.

A person is standing on a beach with a surfboard.

A woman is sitting at a table with a large pizza.

A man is talking on his cell phone while another man watches.

Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard Zemel, Yoshua Bengio, “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention”, ICML, 2015

Ref: A man and a woman ride a motorcycle

A **man** and a **woman** are **talking** on the **road**

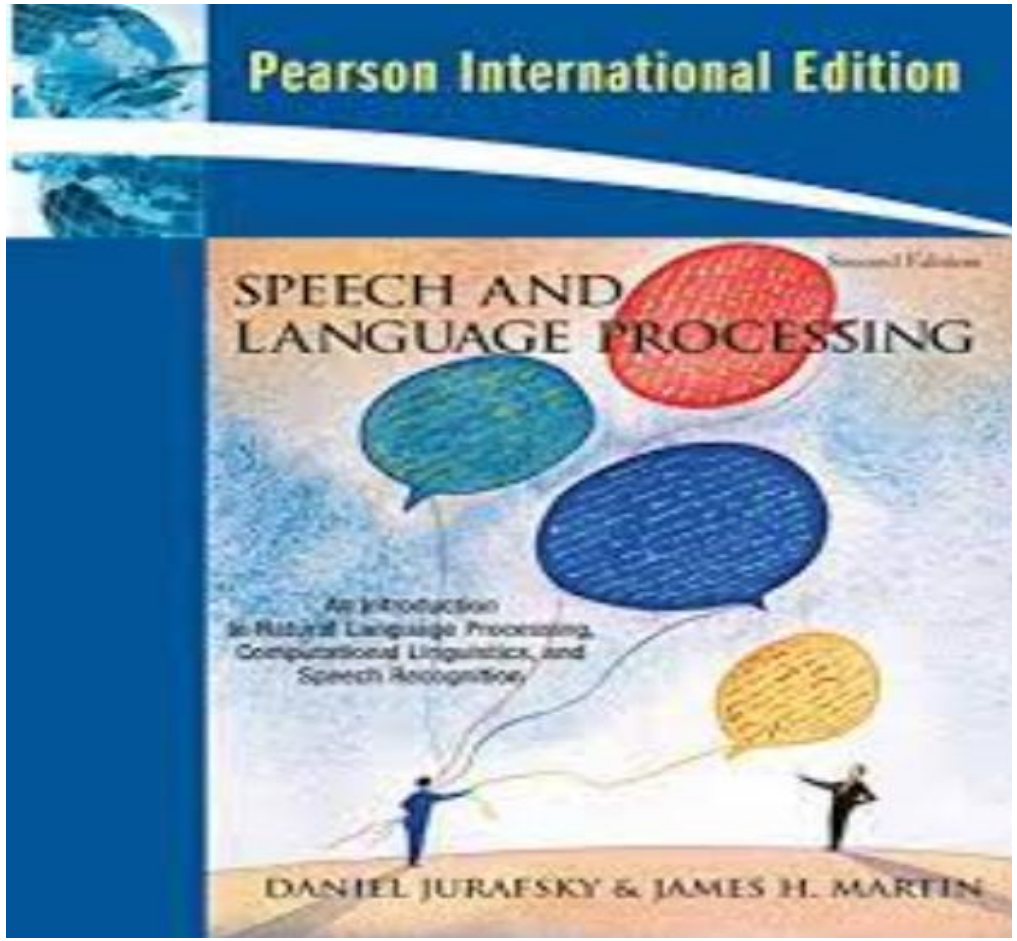
* Possible project?

Ref: A woman is frying food

Someone is **frying** a **fish** in a **pot**

Li Yao, Atousa Torabi, Kyunghyun Cho, Nicolas Ballas, Christopher Pal, Hugo Larochelle, Aaron Courville, "Describing Videos by Exploiting Temporal Structure", ICCV, 2015

Reference



Chapter 9

Question

Thank you